

2025 The International Mathematical Modeling Challenge (IMMC) Summary Sheet

Team Control Number: **US-16158**

Striking a Balance: Fairness and Competition in Sports Scheduling

Whenever we make a decision, we must strike a balance. Sports scheduling is filled with decisions and indecision. Who will host the games? How many games will each team play? Who are teams composed of? What sport are they even playing? At the root of these questions is a central one: how can we create a fair and sustainable international sports league that spans the globe, minimizes travel, and ensures balanced competition? We aim to build an optimized schedule for a worldwide volleyball league where teams are fairly matched, no one is overburdened with travel or games, and every team has time to rest.

To begin, we first identified key components: what sport the Global Sports League (GSL) should focus on, what teams should participate in the GSL, and what considerations we should take into building our sports schedule.

We chose a sport for the GSL by first analyzing Olympic team sport data. This gave us a broader sense of popularity, geographical representation, and competitive equity. To narrow down this list of sports, we compared specific quantifiable factors, including global viewership, economic sustainability, environmental sustainability, and potential for growth. Ultimately, this resulted in choosing women's volleyball.

In choosing the teams that comprise the GSL, we created a criteria of selecting countries that would ensure geographic diversity, logistical feasibility, competitive equity, global expansion, and the overall competitive level. This criteria was built with a foundation in the Fédération Internationale de Volleyball's (FIVB) ranking system. This allowed us to consider relative strengths of teams, which would create a strength ranking system that better reflected the true strength of teams.

The main factors that we considered in creating the schedule were **effectivity, travel distance, number of games, home advantage, time between games, breaks, and attractiveness**.

After identifying the main components of the problem, we broke the problem of scheduling into two main parts. First, we decided who plays whom. Using real-world geographic data, a **mixed-integer linear program (MILP)**, and **Markov Chain analysis**, we built a balanced network of matchups. Each team was assigned a fixed number of opponents, with the goal of minimizing total travel distance while maintaining the fairness of matchups.

Next, we decided when these matches would happen. We developed another MILP program to schedule the games over an 8-month season. This model ensured that each game was assigned a unique day, that no team played more than once per day, and that teams were given proper rest between matches. It also enforced a home-and-away structure so that every team got fair hosting opportunities.

Finally, we developed metrics to measure both the effectivity and attractiveness of a schedule. Effectivity measures how close the final outcome of a tournament is to the "true" skill ranking of the teams, so a lower effectivity score is preferred. Attractiveness generally measures how enticing a certain game is for the audience based on the strengths of the teams playing. Our metrics included a weighted inversion metric for effectivity and intensity, closeness, and entropy for attractiveness. We then simulated multiple different schedule systems to determine the best for our championship round.

We found that a 7-opponent structure strikes the best balance for 20 teams: it spreads travel out fairly while keeping total distance low. Additionally, when expanding to 24 teams, 11 opponents per team became the optimal setup. Our scheduling model successfully generated a full season that respects rest days, limits daily games, and avoids logistical conflicts without any guesswork or heuristics.

Experimentally, we decided that our tournament structure should include a lengthy preliminary round where teams have time to improve against rivals before the top 8 qualify for the championship knockout round. This was more dynamic and lenient than other established schedules.

We used **Monte Carlo** simulations to probabilistically simulate tournament outcomes, giving us insight on different tournament systems. Ending the league with a knockout tournament (in the 20-team GSL league) yielded, on average, an effectivity of 2.5, closeness of 0.55, intensity of 506, and entropy of 0.4. Each of these stats were roughly normally distributed, with the exception of effectivity being skewed right. We performed similar simulations for Swiss and Round Robin tournaments and found knockout to be preferred. Finally, we compared the stats of our schedule to those of the FIVB Women's World Championship schedule, and found an average effectivity of 6-7, closeness of 0.8, intensity of 633, and entropy of 0.56. Though we predict FIVB's schedule will have more attractive games (due to the inclusion of higher-skilled teams), our schedule is more effective than the FIVB schedule while taking into account environmental concerns and geographic diversity.

To the International Multi-Continental Matchmaking Committee

Developing a Volleyball Scheduling System for the Global Sports League (GSL)

Dear Directors of the IMMC,

This is Team 16158 writing to discuss our model and major findings regarding our scheduling model for the proposed Global Sports League (GSL).

After considering a large range of factors, we first determined that women’s volleyball—one of the fastest-growing team sports worldwide—would best serve the IMMC’s hopes of growing the GSL to a global audience while also accelerating the sport’s growth through the support of the GSL, leading to a positive feedback loop.

We then considered our criteria centered on feasibility and representation to choose 20 teams from six continents for our initial schedule.

After this, we considered the different factors to influence our schedule. By the IMMC’s criteria of economic and environmental sustainability and fairness, we decided to consider the additional factors of home advantage, breaks, time between games, effectivity and attractiveness. Generally, our schedule optimizes for fairness by ensuring that all teams have sufficient time for rest and travel between games, and no team plays multiple home games in succession. Further, we ensure our schedule is effective, which means the result of the championship reflect the “true” skill level of the players and that the tournament structure isn’t biased. Finally, we ensure our schedule is attractive to viewers—in particular, that there are intense and unpredictable games to tune into throughout the season (which is sure to bring in great profits!).

The structure of our schedule is indeed enticing to a viewer: for the first eight months of the season, teams play a fixed number of games with their close geographic rivals, allowing fans to support their favorite teams in the area and teams to gain much competitive experience. Then, the top eight rated teams advance to a knockout championship tournament (exciting, right?) where they compete for the top positions. The championship will be the ultimate display of women’s volleyball skill in the world, and the audience (who have found their favorite teams in the eight months prior) will be on the edge of their seats to see the outcome.

It is our hope that our model might be used to create a GSL that will lead to healthy competition for competitors, enjoyment for global fans, and fulfillment for the IMMC.

Thank you for your trust in us.

Best,
Team 16158

OPTIMAL SCHEDULING

Global Sports League



At the **Global Sports League**, we focus on **global team improvement** throughout our **8 month season**---motivated by the ultimate challenge: a **knockout final championship**.

OUR GLOBAL REACH:



20

Different Global
Venues and
National Teams



14

Preseason Games
Played per Team (7
home; 7 away)



6

All Continents
(except Antarctica)
Represented



Pre-Season Game Scheduling

Minimizes distance travelled while allowing for equal and competitive matches.



METHODS OF OPTIMIZING FAIRNESS

1. Minimizing Home/Away Advantages
2. Equalizing Travel and Rest Recovery Time
3. Equalizing Number of Preseason Games

When compared with the Fédération Internationale de Volleyball's World Championship, **our schedule** was found to be **more efficient**, **more geographically diverse**, and **more economically sustainable**.

Our Scheduling Benefits



High-Intensity Matches

Our schedule pushes for intense and close matches that increase economic gains and audience viewership.



Appropriate Point to Skill Reward

Our system is able to structure our schedule such that worse teams do not gain artificial points that don't represent their skill.



Equal Valued Venues

Our schedule is able to connect national teams/venues such that no one venue dominates the international stage.

Competing Countries: Italy, Brazil, USA, Türkiye, China, Poland, Japan, Netherlands, Serbia, Dominican Republic, Thailand, Puerto Rico, Argentina, Colombia, Kenya, Peru, Cameroon, Australia 1, Australia 2

Contents

1	Introduction	5
1.1	Problem Restatement	5
2	Identifying Key Factors (Part 1)	5
2.1	Sport Identification (1a)	5
2.1.1	Variable Definitions	5
2.1.2	Identification Criteria	5
2.2	Team Identification (1b)	8
2.2.1	Variable Definitions	8
2.2.2	Team Criteria	8
2.2.3	Initial GSL Teams	10
2.3	League Schedule Considerations (1c)	10
3	Schedule Creation (Part 2)	11
3.1	Assumptions	11
3.2	Tournament Structure	11
3.3	Mixed Integer Linear Programming (MILP)	13
3.3.1	Variable Definitions	13
3.3.2	Applying MILP	13
3.4	Global Markov Chain Analysis	14
3.5	Analyzing MILP and Markov Chain Results	15
3.5.1	Variable Definitions	16
3.6	Fair Schedule Timing	16
3.6.1	Problem Setting and Notation	16
3.6.2	Binary Variables	17
3.6.3	Constraints	17
3.6.4	Objective and Solving	18
3.7	Schedule Effectiveness and Attractiveness	18
3.7.1	Variable Definitions	18
3.7.2	Effectivity Metric	18
3.7.3	Attractiveness Metric	19
3.7.4	Results	19
4	Expand the Model (Part 3)	21
4.1	New GSL Teams (3a)	21
4.2	Finding the Optimal k-value for 24 Teams	21
4.3	New GSL Schedule	22
5	Generalizing Our Model (Part 4)	23
5.1	Generalizing Rating	23
5.2	Generalizing Matchups	23
5.3	Generalizing Schedule Creation	23
6	Strengths and Weaknesses	24
6.1	Strengths	24
6.2	Weaknesses	24
7	Conclusion	24
8	Works Cited	25
9	Appendix	26
9.1	Problem 1	26
9.2	Problem 2	27
9.3	Problem 3	47

1 Introduction

“Sports can create hope, where there was once only despair. It is more powerful than governments in breaking down racial barriers. It laughs in the face of all types of discrimination. Sports is the game of lovers.” —Nelson Mandela

If music is the universal language, then sport is the universal heartbeat. Few aspects of our shared culture inspire such widespread enthusiasm and passion as sport. In this way, sports bring together billions of people from different continents, people who speak different languages and come from different cultures and have very little else in common except their shared love of the sport.

Ironically, the basis of sport’s unifying power is competition—sport consists of competition. It is essential, then, that we make this competition as legitimate and accessible as possible. This is the goal of the International Multi-Continental Matchmaking Committee’s Global Sports League: to create a competitive environment wherein teams from around the world compete in an equitable and sustainable way.

1.1 Problem Restatement

1. We would like to identify a sport that is ideal for the GSL based on the given factors of competitive equity, economic sustainability, environmental sustainability, growth potential, and geographic diversity. We must identify additional factors for consideration in building a schedule for the GSL in this sport.
2. We are tasked with creating a mathematical model to create a full schedule for 20 teams in the GSL over 8-9 months. The model should emphasize fair treatment of teams, equitable travel time/distance for all teams, and balanced competitiveness. After creating a model and generating our schedule, we evaluate how well it follows the IMMC’s criteria and determine the number of games each team must play to determine a winner.
3. We must extend our 20-team schedule to 24 teams, choosing 4 extra teams that emphasize geographic diversity. We must then alter our model to consider the 24 team schedule and report any changes to the schedule generation procedure and number of games played to determine a winner.
4. Broadly discuss how we could generalize our scheduling model to different sports, considering different rule sets, nature of the sport, and team dynamics.
5. We must communicate our solution to the IMMC, persuading them of the legitimacy and completeness of our solution to convince them how our model connects to the IMMC’s goals. This should be paired with a one-page visual detailing our schedule for a wide audience.

2 Identifying Key Factors (Part 1)

2.1 Sport Identification (1a)

2.1.1 Variable Definitions

Table 1: Variable Definitions

Variable	Definition
B_C	Number of bronze medals won per country
S_C	Number of silver medals won per country
G_C	Number of gold medals won per country
P_C	Total points earned per country
M_S	Number of countries that acquire a medal in each sport

2.1.2 Identification Criteria

The International Multi-Continental Matchmaking Committee (IMMC) seeks to uphold several principles in administering the Global Sports League (GSL). To do this, it must choose to support a sport that is consistent with these principles, which can be summarized in four criteria: **competitive equity**, **economic sustainability**, **environmental sustainability**, and

potential for growth. We also consider geographic diversity, but because this consideration is part of our analyses of the three listed criteria, we do not make geographic diversity a separate criterion. See Figure 1 for our identification criteria.

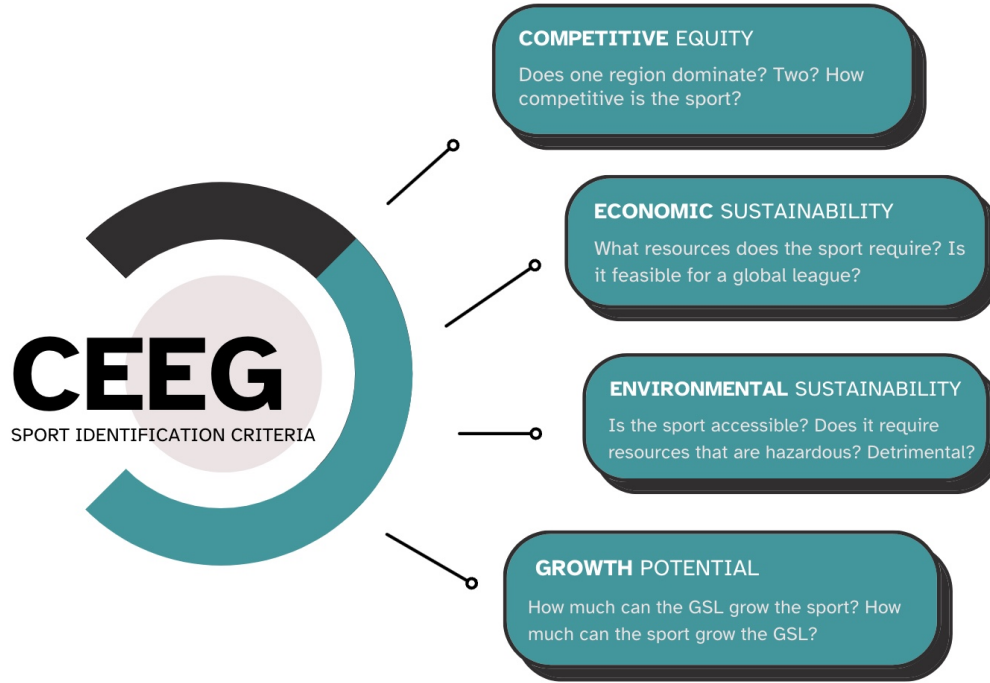


Figure 1: Sport Identification Criteria. Note that geographic diversity isn't portrayed as a standalone criterion since it is considered throughout our other criteria.

We first identify sports that satisfy the criterion of competitive equity. Then, we identify the best of those choices through the lenses of economic sustainability, environmental sustainability, and potential for global growth.

We begin the analysis of competitive equity with the set of team sports included in the Olympics. The reason for using this set is that the criteria for including a sport in the Olympics^[1] closely mirror the considerations we must make when choosing a sport for the GSL, including global reach, well-established rules, and ethics—further, this is a dynamic set of sports, as the Olympics has adapted to global trends over the course of its over-century-long history, making it suitable to draw from to create a contemporary league. We note that this set of sports has the potential to diverge into subjective territory because host countries wield some influence over which sports to include in their Olympics, possibly introducing some regional bias. However, this has only been in effect since 2020, and our data goes up to 2016.

We analyze Olympic data since 1980^[2] to find team sports that are fairly popular and have enough global data for us to analyze competitive equity and geographic diversity. We weigh each medal differently to account for the difference in value of each medal type by using a point system in which bronze medals are equivalent to 2 points, silver to 3 points, and gold to 6 points. All weighting systems are arbitrary to an extent, but we choose this 2-3-6 weighting for bronze, silver, and gold medals based on Zipf's Law, which is that the value of an object is often inversely proportional to its rank.^[3] We accumulate the total number of points with the following equation (see Appendix A for code):

$$P_C = 2B_C + 3S_C + 6G_C$$

Now that we have a way to quantify the performance of a country in a given Olympic sport, we apply a standard deviation to each sport to analyze the point distribution among countries. We analyze this data to find if there is a disparity in the competition for a sport: a high standard deviation indicates that one country has historically dominated the sport while a lower standard deviation indicates that the skill level between countries is relatively even. The ideal sport for the GSL would have a low standard deviation, as that sport would be conducive to a competitive league. We interpret this to mean that the sport demonstrates competitive equity. In addition, if we define a sport demonstrating geographic diversity as one in which, historically, teams from many different countries have found success (in other words, sports for which no countries dominate the sport), this definition becomes very similar to that of competitive equity. So, the sports we identify as demonstrating competitive equity also demonstrate some extent of geographic diversity by this definition. The results per sport can be seen below in Table 2:

Table 2: Top 5 Smallest Standard Deviations

Sport	STD
Rugby	21.53
Water Polo	97.03
Softball	97.21
Handball	113.22
Football (Soccer)	115.38

However, we find that these results do not accurately account for geographic diversity, as certain sports, such as rugby, have low standard deviations but very little diversity in winning teams; only 6 countries have medaled in rugby since 1980, while 19 countries have medaled in handball over the same period. To account for this, we include a scaling factor of $\frac{1}{(M_S)^2}$. We choose a quadratic scaling factor as it more heavily emphasizes the effect of geographic diversity. The results can be seen below in Table 3:

Table 3: Top 5 Smallest Scaled Standard Deviations

Sport	Scaled STD
Football (Soccer)	0.22
Handball	0.31
Water Polo	0.43
Rugby	0.60
Volleyball	0.75

Now, having accounted for competitive equity, we consider the remaining criteria of economic and environmental sustainability.

We begin with economic sustainability. We use global viewership as a proxy for economic sustainability from the perspective of the GSL—sports leagues rely on viewership, as their business models are based around media contracts,^[4] the values of which are very closely related to viewership. The percentage viewership for each of the sports are shown in Table 4:^[5]

Table 4: Global Viewership

Sport	Viewership Share
Football (Soccer)	56%
Volleyball	17%
Rugby	6%
Handball	<3%
Water Polo	<3%

Having considered the economic sustainability from the perspective of the GSL, we also consider the extent to which these sports are economically accessible for potential athletes in countries that have historically been underrepresented in those sports, as it is reasonable to assume that easier access to professional-level equipment and facilities would have a positive effect on the upward mobility of the performances of these countries in those sports. More specifically, water polo stands out as being particularly unsustainable, while volleyball is particularly sustainable. Water polo requires deep pools with very specific dimensions,^[6] which are costly to construct and maintain; volleyball and handball, in contrast, require facilities and equipment with comparatively minimal expenses, avoiding the costs of managing outdoor grass fields.

These environmental costs also closely reflect the economic costs: the high costs of maintaining pools^[7] and grass fields^[8] lead to environmental consequences, while volleyball and rugby are comparatively sustainable because the equipment and facilities do not have any qualities that make them particularly damaging to the environment.

The “potential for growth” criterion is twofold: we consider both the potential for the sport to grow the GSL and the potential for the GSL to grow the sport. We have already shown through our analyses of equity, economic sustainability, and environmental sustainability that volleyball is a suitable sport to base the GSL on: volleyball is the sport that most consistently meets these criteria. With regard to the popularity of volleyball, women’s volleyball is already doing extremely well, especially among young people—game attendance and viewership has exploded within the past several years. Even with the current success of the FIVB’s Volleyball Nations League (VNL) in growing the sport, however, the GSL also a lot to offer—the VNL is only five weeks long, leaving international volleyball fans without exciting matches for the remainder of the year. The GSL, in contrast, is approximately eight months long, which could help build competitive storylines throughout the season and build more loyal fanbases, similarly to the National Basketball Association, which is approximately six months long.

For these reasons, **women’s volleyball is the most consistently well-suited sport for the GSL.**

2.2 Team Identification (1b)

2.2.1 Variable Definitions

Table 5: Variable Definitions

Variable	Definition
S_i	Score before game
S_f	Score after game
K	Match importance
n	Index: 1 for 3-0 win, 2 for 3-1 win, 3 for 3-2 win, 4 for 2-3 loss, 5 for 1-3 loss, 6 for 0-3 loss
Δ	Scaled difference in ranking between two teams A and B : $\frac{8(S_A - S_B)}{1000}$
C_i	Cut points for each index, which are experimentally derived from FIVB data: $C_0 = -\infty$, $C_1 = -1.06$, $C_2 = -0.394$, $C_3 = 0$, $C_4 = 0.394$, $C_5 = 1.06$, $C_6 = \infty$
P_i	Probability distribution for a given event: $\Phi(C_i + \Delta) - \Phi(C_{i-1} + \Delta)$
Φ	Normal Cumulative Density Function
R_i	Result: $R_1 = +2$, $R_2 = +1.5$, $R_3 = +1$, $R_4 = -1$, $R_5 = -1.5$, $R_6 = -2$
E	Expected result: $\sum_{i=1}^6 R_i P_i$

2.2.2 Team Criteria

In creating the GSL, the IMMC hopes to balance competitive equity with geographic diversity in selecting teams to be featured on the global level. Since the Fédération Internationale de Volleyball (FIVB) has been hosting international games since 1947, we will utilize their established network of global team rankings.

The FIVB ranking system is a point system in which teams gain points after winning games and lose points after losing games. It is based on the probability of winning games—the less likely a team is to win a game, the more points they gain for winning and the fewer points they lose for losing; conversely, the more likely a team is to win a game, the fewer points they gain for winning and the more points they lose for losing. The system calculates probability distributions by using cut-points, or points on the normal distribution that separate regions of probability for each outcome. See Figure 2 below for a graphical representation of the outcomes between two equally skilled opponents:

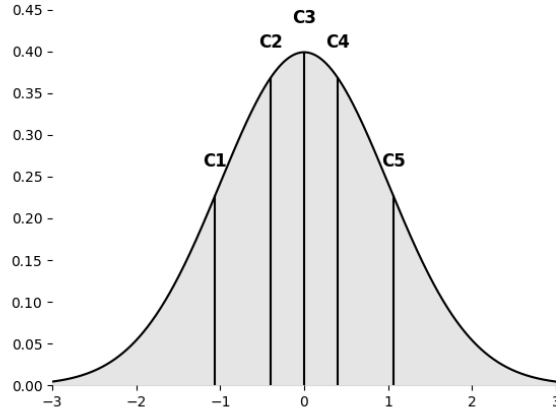


Figure 2: Cut-Points (Note that C_0 and C_6 do not appear)

The region between C_0 (at $-\infty$) and C_1 is the probability of a 3-0 win, the region between C_1 and C_2 is the probability of a 3-1 win, and so on (see 2.2.1 for a full list of outcomes and corresponding cut-points). The graph is symmetric only because it represents the probability distribution of games between two equally skilled opponents: if one team were more skilled than the other, the cut-points would shift left or right to reflect the change in win probability for each team.

The number of points that are redistributed per game are calculated according to this formula:

$$S_{\text{after}} = S_{\text{before}} + \frac{K(R - E)}{8}.$$

Here, R is the actual result of a game, where $R_1 = +2$, $R_2 = +1.5$, $R_3 = +1$, $R_4 = -1$, $R_5 = -1.5$, and $R_6 = -2$, so $R - E$ represents the signed difference between the actual outcome and the predicted outcome (similar to a residual). In addition, K represents the weight of the match, or how significant the match is. We assign a value of $K = 45$ throughout the tournament, as we would like each match to be weighted equally.

The FIVB World Ranking^{[9][10]} will play a major role in the selection of teams for the GSL. This can be seen within the following criteria:

1. **Selected teams must have the highest ranking points in their respective continents based on international tournaments hosted by the FIVB.**

Justification: This ensures a high competitive level while also allowing for logistical feasibility, as teams competing in the FIVB have resources and are established within the sporting world.

2. **Selected countries must have a venue to host international tournaments.**

Justification: This ensures logistical feasibility, as it guarantees that countries within the GSL are able to host tournaments.

3. **A country can have at most one national team that represents them in the GSL.**

Justification: This ensures that we are able to support more geographic diversity through global representation.

4. **There must be at least two teams represented per continent.**

Justification: This is another measure of ensuring a level of geographic diversity while also helping to improve the culture and overall competitive level of volleyball globally. Australia will be allowed two national teams to satisfy criterion #4. We make an exception here for criterion #3, as Australia is the only country from Oceania competing in the FIVB.

5. **There can only be at most four teams per continent.**

Justification: This allows continents with high performers to send more teams to the GSL while ensuring that one continent does not dominate in the league.

2.2.3 Initial GSL Teams

After selecting teams through our listed criteria in 2.3.2, we choose correlated venues based on popularity. Since the most popular venues are often closest to metropolises, they offer high viewership and logical feasibility as they are accessible locations within a country.

Table 6: 20 Selected Teams for the Global Sports League

Rank	Team	Continent	Venue	FIVB Points
1	Italy	Europe	PalaLottomatica	437.03
2	Brazil	South America	Arena da Baixada	407.09
3	USA	North America	American Sports Center	362.27
4	Türkiye	Asia	Sinan Erdem Dome	352.61
5	China	Asia	Beijing National Stadium	350.30
6	Poland	Europe	Atlas Arena	349.75
7	Japan	Asia	Ariake Arena	325.18
8	Canada	North America	Rogers Centre	284.76
9	Netherlands	Europe	Johan Cruyff Arena	283.99
10	Serbia	Europe	Taşmajdan Sports and Recreation Center	280.42
11	Dominican Republic	North America	Centro Olímpico Juan Pablo Duarte	260.54
12	Thailand	Asia	Indoor Stadium Huamark	194.91
13	Puerto Rico	North America	Coliseo de Puerto Rico José Miguel Agrelot	182.58
14	Argentina	South America	Estadio Luna Park	180.96
15	Colombia	South America	Estadio Metropolitano Roberto Meléndez	152.37
16	Kenya	Africa	Kasarani Indoor Arena	152.15
17	Peru	South America	Estadio Nacional del Perú	93.05
18	Cameroon	Africa	Yaoundé Multipurpose Sports Complex	85.69
19	Australia	Oceania	Accor Stadium	44.13
20	Australia	Oceania	Accor Stadium	44.13

2.3 League Schedule Considerations (1c)

Based on our research,^[11] established literature on comparing tournament designs mainly consider four criteria: **effectivity/efficacy**, **fairness**, **attractiveness**, and **strategy-proofness**. We are asked to take into account travel distance, number of games played, and equitable matchups to create an ideal format and schedule.

We consider the effectivity/efficacy (which we will henceforth refer to as effectivity) of a schedule to be the extent to which the outcome of the tournament reflects a predetermined, objective ordering of teams based on skill. For this objective ordering, we use the FIVB rankings mentioned earlier, as points in the FIVB ranking are cumulative over both major and minor volleyball tournaments over several years. The results of an effective tournament structure will generally follow the strengths of the teams, with some differences that reflect fluctuations in performance. We will discuss effectivity in more detail in later sections.

The IMMC’s criterion of equitable matchups align with the criterion of fairness. The equity of a matchup is subjective, and the matchups created according to one definition of equity will be different from those according to another definition. We will specify our definition of equity for our purposes by outlining the factors our team most values. First, the scheduling should not directly disadvantage any team—for example, by scheduling two away games (where significant travel is needed) in close succession, which would lead to compounding effects of jet lag and fatigue. It should also not give an advantage to

any team—for example, by having one team play only home games or having considerably less travel time than other teams. We consider three factors that constitute a schedule with equitable matchups: **home advantage**, **time between games**, and **breaks**.^[12] We define a break as a point in the schedule where a team either plays two home games in a row or two away games in a row. In our case, two home games in a row can potentially lead to a real advantage, since the team gets the opportunity to recover in one location.

Another aspect we consider is the carry-over effect. It is reasonable that the result of a previous game would impact the next game in some way (team morale, fatigue, etc). However, studies have shown that carry-over effects generally do not have a statistically significant impact on the outcome of matches.^[13] Also, due to travel, games will be scheduled multiple days apart. For these reasons, we do not consider the carry-over effect when making our schedule.

The attractiveness criterion refers to, in some sense, the intensity of a match, which is directly related to audience engagement and economic profit from the match. This is usually maximized when the two teams in a match are both at a high skill level—audiences generally prefer to see high-level competitive games than lower-level competitive games or unbalanced games where one team is much stronger than the other.

Finally, any tournament schedule should be strategy-proof. This means that teams should have no incentive to act in any way except to try their hardest to maximize their chances of winning every match they play at all points throughout the tournament. This criterion is most often considered when determining prize allocation, and since we are not considering that in our schedule creation, there are effectively no external incentives, and every team solely aims to win the championship.

In summary, our schedule creation will consider **effectivity**, **travel distance**, **number of games**, **home advantage**, **time between games**, **breaks**, and **attractiveness**.

3 Schedule Creation (Part 2)

3.1 Assumptions

1. We assume teams play their scheduled games.

Justification: There are any number of reasons a team could miss a game—flight cancellations, for instance—however, for the sake of developing a schedule, we assume that all teams to play their assigned games.

2. We assume teams always try their hardest during matches.

Justification: We want to assume a general consistency of performance for teams. We will be utilizing the strength of a team as their FIVB score, so we want their performance to be close to the level indicated by the FIVB score. Of course, teams have space to improve or get worse between games which will end up changing their FIVB score, but teams cannot perform at a drastically different level than indicated by their FIVB score (for example by hiring professional players from other countries). Note that this is

3. We assume countries have sufficient resources to send their team(s) to scheduled games.

Justification: Since these countries compete in the FIVB, they have enough resources to train internationally competitive teams and send them to international competitions, but we assume this holds for any games our schedule assigns for teams so that teams can play all games.

4. We assume teams are treated equally in hosting locations.

Justification: We are attempting to control fairness in scheduling, but have no control over how teams are treated at hosting countries. We'd like all countries to uphold values of fairness and equality and not give any special treatment that would affect the outcome of matches (again, since we are using FIVB scores as a proxy for how a team will perform at any given match).

3.2 Tournament Structure

Creating a schedule is inherently an optimization problem of all of the aspects we value. It is impossible to optimize every factor we consider since some are in direct opposition with one another. For example, we would like to expand the GSL and thus host popular games, but every additional game requires more travel time and therefore has an adverse effect on the environment.

There are a few important tournament styles that influenced our thought process in choosing an initial schedule: **knock-out**, **round robin**, and **Swiss**. Knockout schedules are the standard tournament bracket, where a team is eliminated after a loss and the winner of the tournament is the team with 0 losses. In round robin, every team plays each other team exactly once. Swiss style tournaments are a variant of round robin where teams don't play every other team; instead, teams always

face teams with a similar running score to them. These are the basic version of these systems, as all of them can be extended (for example, double knockout and double round robin allow teams to play more games). In particular, some of these systems are preferable to others depending on what is valued more. It has been shown that round robin tournaments are more effective than knockout tournaments,^[11] but this comes at the cost of having to play more matches. In addition, knockout tournaments generally end in very intense matchups that are very popular, while round robin leagues usually do not have such climactic endings. Swiss tournaments are somewhat of a middle ground, but the outcome of the tournaments depend heavily on scoring metrics, making them much more dynamic than the other systems.

The GSL emphasizes fairness in its scheduling. This means we should ensure we can get as close as possible to competitive equity in our scheduling as possible. We would like each team to have the same number of breaks, especially for streaks of home games. The home advantage is a real and well-studied phenomenon, and in particular, it matters for fairness how home/away games are assigned skill-wise. Consider a weak team in a round robin tournament playing 8 games. It plays 4 home games and 4 away games, but its home games are scheduled against similarly skilled teams while the away games are against stronger teams. The home advantage benefits this team in particular since it boosts its chance of winning against similarly skilled teams, while the decrease in win probability does not matter much against the stronger teams. Therefore, we would like to balance the aggregate skill level of the teams faced at home and away games. If team i is facing teams $1, 2, \dots, i-1, i+1, \dots, n$ (where $i < j$ means i is a stronger team than j), then the home/away assignments should alternate skill-wise, so team i could play away against team 1, at home against team 2, and so on. This is called a ranking-fair schedule, and we can guarantee it exists with each team only having one break when there are $n = 4k$ teams in a round robin tournament.^[12] We will try to mimic this construction in the format we end up deciding for the GSL.

To try and account for all of the factors we value, we decided to initially consider a hybrid tournament style. The GSL runs over 8-9 months, so we decide to split it into two stages. The first stage takes place over the first 7-8 months, consisting of preliminary rounds where teams play each other to increase their rating (using the FIVB rating system). The second stage is made up of the top 8 teams by rating established during the first stage, which then compete in a championship knockout bracket. The preliminary stage aims to maximize consistency and establish an effective ranking, while the inclusion of a final championship keeps audiences engaged and wanting to see the final intense games.

The preliminary round functions similarly to a round robin system, as this gives teams a good chance at increasing their rating. However, we would also like to reduce travel distance due to environmental concerns, so teams will only play the (roughly) k nearest teams to them. The preliminary round and final championship can be viewed in Figure 3.

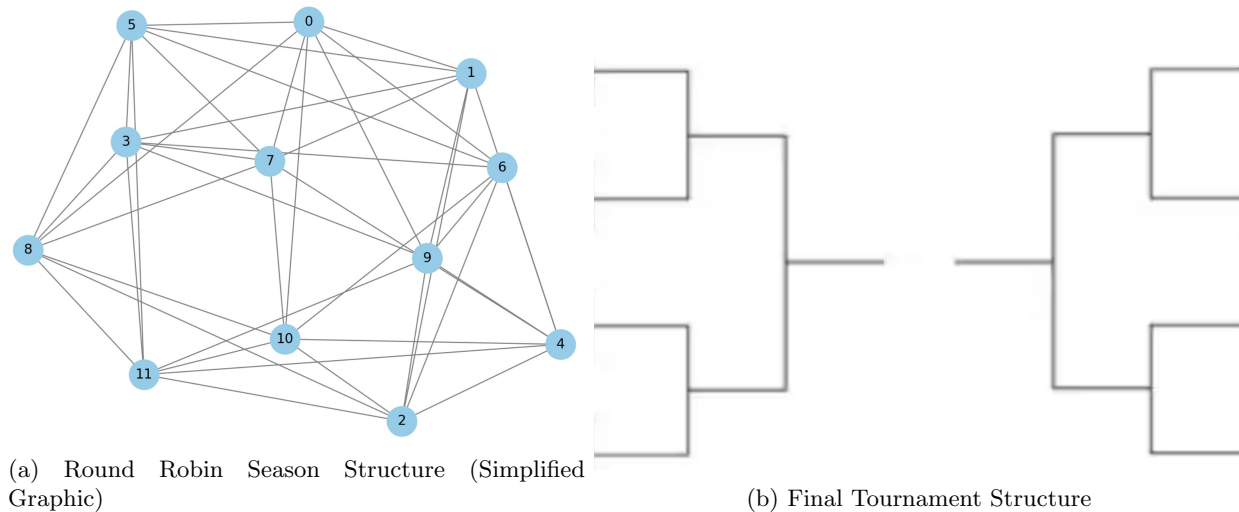


Figure 3: Tournament Structure for GSL

Formally, we can frame this in the language of graph theory. If every team is a node and every match played is an edge, the preliminary round should be a k -regular connected graph (every node has degree k). We want to make sure groups of teams are not isolated from the rest of the pool, so the graph must be connected (and there should not be ‘dependency’ points, which we will discuss later). For example, if it was impossible for an Australian team to play with a team that faced Italy, it would be impossible for Australia’s rating to exceed Italy’s, which is not desirable. We will decide the value of k that balances minimizing total travel distance while allowing teams to face a variety of skill levels. After this, we will build the schedule over time so that home/away games and time between games are fair across all teams. The preliminary round feeds into the championship tournament, which will initially be a single knockout tournament of the top 8 ratings. We will develop models to evaluate the effectivity and attractiveness of a schedule and apply it to our initial schedule, alter our schedule accordingly, and compare it to the international FIVB schedule.

3.3 Mixed Integer Linear Programming (MILP)

3.3.1 Variable Definitions

Table 7. Variable Definitions

Variable	Definition
k	Number of teams played by each team in preliminary round
d_{ij}	Distance between venues i and j
x_{ij}	Binary Variable to indicate team i and j matchup ($i < j$)

3.3.2 Applying MILP

First, we want to find the number of teams each team faces in the preliminary round by balancing total distance traveled and ensuring there is no dependency. What we mean by dependency is that, for a lower value of k it is possible we could have a connected graph but the connectivity heavily on one team, so any transitive rating updates would have to occur through only a few teams. We would like to reduce this since that makes it less probable that teams will actually be able to dynamically change their rating throughout the preliminary season. Mathematically, we use **Mixed Integer Linear Programming (MILP)** to ensure each team faces k teams (we have a k -regular graph) while minimizing total distance. MILP is a type of optimization problem where the objective function and constraints are linear, but some or all decision variables are restricted to be integers (often binary). In contrast to standard linear programming (LP), where decision variables are reals, MILP problems are generally NP-hard due to the integrality constraints.

We have a total of $n = 20$ teams. Let them be indexed $i = 0, \dots, 19$. We want to create a k -regular subgraph on the 20 teams, where k is the number of teams each team plays. For each pair of teams (i, j) , we will start by precomputing the geodesic distance, d , using the geopy library. The longitude and latitude coordinates of our venues can be found in the appendix.

$$d_{ij} = \text{dist}[i][j] \text{ (in km)}$$

$$d_{ij} = d_{ji}$$

$$d_{ii} = 0$$

We use a binary variable $x_{ij} \in \{0, 1\}$, where $i < j$, to indicate if teams i and j play each other.

$$x_{ij} = \begin{cases} 1 & \text{if edge } \{i, j\} \text{ is "picked"} \\ 0 & \text{otherwise} \end{cases}$$

Our objective is to minimize total travel distance. As an example, let us start by assuming $k = 4$. We want a 4-regular graph (each team plays exactly 4 opponents). The sum of all chosen pairwise distances of the 4 edges incident to each node should be as small as possible. Therefore, the objective function is

$$\min \sum_{0 \leq i < j < 20} d_{ij} x_{ij}$$

We use the PuLP python library to implement the integer linear programming system into code.

```
1 model += pulp.lpSum(dist[i][j] * x[(i,j)])
2     for i in range(n) for j in range(i+1,n))
```

Our constraint is the degree at each node. Since we only have variables x_{ij} for $i < j$, for a fixed i we sum over all $j \neq i$ by using $\min(i, j)$ and $\max(i, j)$ to ensure the variables are in the correct order.

$$\sum_{j \neq i} x_{\min(i,j), \max(i,j)} = k$$

$$\forall i \in \{0, \dots, 19\}$$

```
1 for i in range(n):
2     model += (
3         pulp.lpSum(
4             x[(min(i,j), max(i,j))])
5             for j in range(n) if j != i
6         ) == 4
7     )
```

Now that we have our integer program, we will solve it with COIN-OR Branch and Cut (CBC), which is an open-source solver for mixed-integer linear programming. The CBC solver starts by relaxing the integer constraint on the ILP, which is called LP Relaxation, to find the optimal solution, which is not necessarily integral. If the optimal solution is not integral, the CBC solver performs cuts or branches, which is meant to split up the problem into subproblems to attempt to solve for integer solutions. After `model.solve()` is run, each instance of the code

```
1 if pulp.value(x[(i,j)]) > 0.5:
2     adj[i,j] = adj[j,i] = 1
```

marks edge (i, j) in the k -regular network. We then use this to build and print the full 20×20 adjacency matrix.

3.4 Global Markov Chain Analysis

We must then identify which k is the best as it is unfeasible to have a $k = 19$ schedule system. Such a system is displayed in Figure 4.

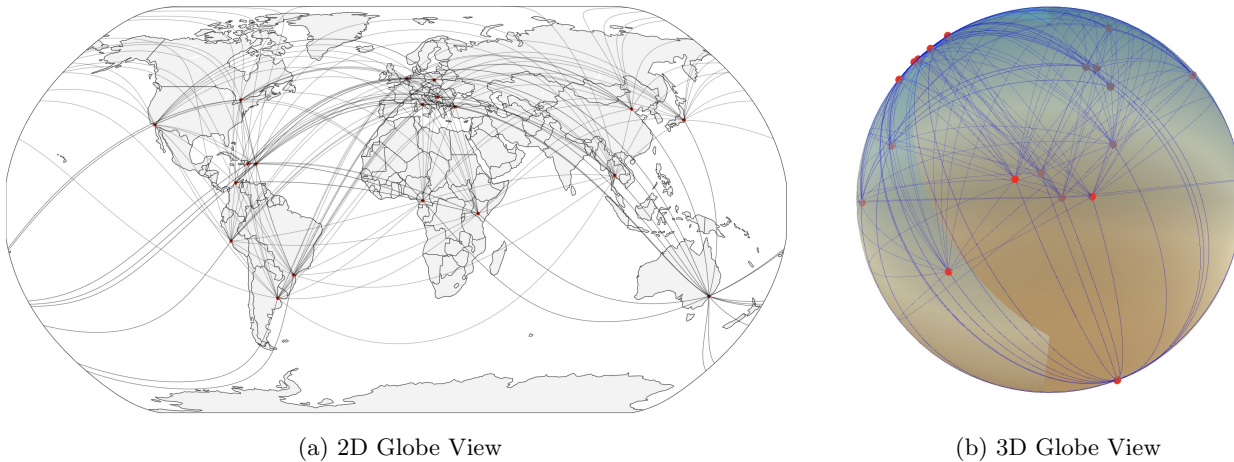


Figure 4: Venue Connections for $k = 19$

To identify which k is the best, we will run through each k from $k = 1$ to $k = 19$ to construct a transition matrix for each. We will then find the steady state solution of the transition matrix and find standard deviation between the entries. The standard deviation between the entries will tell us how uneven the steady-state probabilities are across venues. A small standard deviation indicates that venues are visited with roughly equal frequency, which we want. Meanwhile, a large standard deviation means that some venues dominate in visitation probability. Obviously, if $k = 19$, our standard deviation is minimized as then each node would have equal chances of being visited. However, this would make the total distance traveled maximized. We want a balance between minimizing the standard deviation and minimizing the total distance.

Using $k = 4$ as an example, the 20 by 20 adjacency matrix we get from solving the MILP is shown in the appendix. We now need the transition matrix, P , which we can easily get by normalizing each row of the adjacency matrix. Next we need to find the steady state solution to this transition matrix.

$$\pi^T P = \pi^T \iff P^T \pi = \pi$$

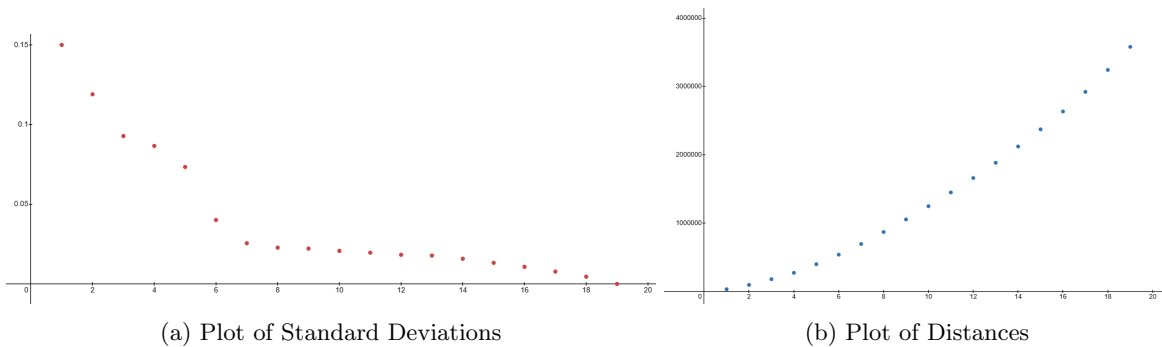
So, we are looking for the eigenvector associated with the eigenvalue of 1 for P^T . We know $P^T v = \lambda v$. We will find the needed eigenvector computationally using Python's numpy library. Due to numerical precision, the eigenvalue computed may not be exactly 1, but very close (e.g., 0.99999). Thus, our code finds the eigenvalue closest to 1 numerically and then gets the associated eigenvector. Next, our code checks to make sure that the eigenvector is normalized before proceeding to find the standard deviations between the entries. For $k = 4$, the standard deviation turns out to be 0.08660. The total distance needed to be traveled can be computed just by summing up all the distances along edges formed by our graph. For $k = 4$, this distance is 271,266 km. We now run through this same process for all possible k ($k = 1$ to $k = 19$). The results of this are displayed in the table below.

$k = 7$ appears to be the optimal choice for k . We have reduced the standard deviation from 0.15 to 0.0255 (83% decrease) at a cost of 694,000 km. Notice that on the plot of standard deviations in Figure 9, we will have diminishing returns as the value of k gets larger.

Going from $k = 7$ to $k = 8$ only drops the standard deviation by 10% but adds 175,000 km. Intuitively, $k = 7$ looks like the knee point of the standard deviations plot, which is a signal to stop adding edges, given that our distance plot appears

Table 7: Standard Deviation and Distance Sum for Different k values

k	std()	Distance (km)	k	std()	Distance (km)
1	0.15000	307757	11	0.01964	1448963
2	0.11902	94036	12	0.01831	1660710
3	0.07364	178182	13	0.01784	1883307
4	0.08660	271266	14	0.01586	2122105
5	0.07341	396814	15	0.01320	2373747
6	0.04016	537591	16	0.01078	2635309
7	0.02557	693781	17	0.00791	2922969
8	0.02283	869172	18	0.00419	3243786
9	0.02219	1053176	19	0.00083	3582378
10	0.02077	1247309			

Figure 5: Steady state analysis dependent on k

to be quadratic or exponential. One note is that we initially started by choosing the seven closest teams to each venue. However, it is not possible to always choose the nearest seven venues from any one node. Suppose for each team (vertex), we greedily select the seven closest other teams based on geographic distance and added directed edges to them. This builds a directed adjacency matrix where Team A includes B if B is near A, but this does not guarantee that B includes A in its list. This creates an asymmetric graph. So, while A might consider B one of its closest neighbors, B might not consider A as one of its closest. Also, we would have uneven degrees as some teams are selected by many others (degree > 7) while others are not selected by enough edges (degree < 7). Thus, this approach leads to both an asymmetric and a non-regular graph. So, we will use the transition matrix generated by $k = 7$. This system is displayed below in Figure 6

3.5 Analyzing MILP and Markov Chain Results

By choosing k that minimized the standard deviation and distance, we were able to balance the distance traveled with the interconnectedness of each venue.

Minimizing distance has obvious benefits for logistical feasibility as we are able to reduce the amount of money spent by each team in the GSL, the amount of time between matches, and maximize viewership with smaller time zone differences. It then follows that there will be a smaller environmental impact as less travel implies less carbon emissions and other detrimental effects.

By minimizing the standard deviation, we were able to account for the interconnectedness of venues. We defined interconnectedness to mean the variability of rating of each venue's home, national team. A low standard deviation indicates that there is a high fluidity (inflow and outflow) at a venue. This is important because if there is a high standard deviation, this would indicate that we have closed ranking loops. For example, since Australia was further away from most of the other teams, we were worried that Australia would not be able to increase their ranking significantly as they wouldn't have the opportunity to play and be affected by as many teams in our distance minimized schedule. Meanwhile, teams like Italy and the USA that are much closer to other teams would have the opportunity to change ratings. Additionally, they would create closed loops of high ratings since the FIVB ranking is dependent on relative strengths, which would eventually break the GSL into regions of high and low rating. Thus, by minimizing standard deviation, we decrease the effect of geography in the GSL

Figure 6: Venue Connections for $k = 7$

and allow for more rating variability.

This also leads into the idea of the page rank algorithm which allows the importance of websites to be revealed—in our case, the importance of the venues. By minimizing the standard deviation, we are minimizing differences in the values of venues and maximizing interaction which improves global equity and overall growth of Volleyball in the GSL.

3.5.1 Variable Definitions

Table 8. Variable Definitions

Variable	Definition
T	Set of venues (teams)
E	Undirected regular-graph edges (opponent pairs)
O	All game “occurrences”, two per pair, encoding home/away order.
$x_{(i,j,k),d}$	Binary decision variable to indicate if k-th meeting of a pair (i, j) is on day d
$h_{(i,j,k)}$	Binary variable to indicate which team has the home game
D	The set of days allotted for games

3.6 Fair Schedule Timing

We again use MLP to turn the matches we have into a real (time-based) schedule. In particular, we want to enforce that there is sufficient time between games for every team for rest and travel to uphold the IMMC’s criteria of fairness. We take the preliminary round to be over 240 days, or approximately 8 months.

We consider several details about the optimal home-away split. We already have a 7-regular graph on 20 nodes, but essentially we turn every edge into two edges (two games per matchup) and make them directed (to indicate home/away status per game). What we want to ensure is that, for any given team, the aggregate skill level of the teams they play away and the teams they play home against are roughly equal. The best way to do this is to alternate home and away skill-wise (so team i plays team 1 away, team 2 at home, etc). This constraint should be paired with the fact that we want home and away to alternate with respect to time as well, as we want to eliminate any chance of a home advantage somehow stacking for teams. These constraints ended up being too constricting for the model, but the final schedule reflects these ideas nonetheless.

3.6.1 Problem Setting and Notation

Venues (Teams) Let $T = \{1, \dots, 20\}$ represent our 20 venues (teams). We have a fixed simple, undirected, 7-regular graph (the matchups we found previously) defined as:

$$E = \{(i, j) : i < j, \text{ venue } i \text{ plays venue } j\}, |E| = 70, \deg(i) = 7 \forall i \in T$$

Days We define a scheduling horizon of $D = \{1, 2, \dots, 240\}$, representing an 8-month season discretized into 240 equal slots.

Occurrences Each pair $(i, j) \in E$ meets exactly twice (alternating home and away). We index these two meetings by $k \in \{0, 1\}$, creating the set of games to be scheduled as follows:

$$O = \{(i, j, k) : (i, j) \in E, k = 0, 1\}, |O| = 140$$

3.6.2 Binary Variables

Similar to the MILP from earlier, we will introduce a binary decision variable $x_{(i,j,k),d} \in \{0, 1\}$.

$$x_{(i,j,k),d} = \begin{cases} 1 & \text{if the } k\text{-th meeting of pair } (i, j) \text{ is placed on day } d \\ 0 & \text{otherwise} \end{cases}$$

$$(i, j, k) \in O, d \in D$$

In addition, we introduce a “Home” flag that is defined as follows:

$$h_{(i,j,k)} \in \{0, 1\} \begin{cases} 1 & \text{if in meeting } (i, j, k), \text{ team } i \text{ is at home} \\ 0 & \text{if team } j \text{ is at home} \end{cases}$$

We use these binary variables when setting constraints for the MILP model.

3.6.3 Constraints

We chose six constraints for this model.

Cover Each Occurrence Exactly Once Every required game must be placed on exactly one day. We can represent this as:

$$\forall (i, j, k) \in O : \sum_{d \in D} x_{(i,j,k),d} = 1$$

No Double-Booking A venue t can not host more than one game on the same day. This can be represented as:

$$\forall t \in T, \forall d \in D : \sum_{\substack{(i,j,k) \in O \\ i=t \vee j=t}} x_{(i,j,k),d} \leq 1$$

One Home/Away Game per Team in a Pair For each undirected pair (i, j) , we want one occurrence with i home and the other with j home. This can be represented as:

$$h_{(i,j,0)} + h_{(i,j,1)} = 1$$

Ordering the Two Meetings We want to ensure the alternating home/away sequence by ordering occurrences. We have an external rule that “occurrence $k = 0$ ” must come strictly before “occurrence $k = 1$ ” for each pair, to lock in our alternating home/away sequence. We encode this by equating each scheduled day to its weighted sum and requiring:

$$\forall (i, j) \in E : \sum_{d \in D} dx_{(i,j,0),d} + 1 \leq \sum_{d \in D} dx_{(i,j,1),d}$$

As an example, let us say the first meeting is on day d_0 and the second day is on d_1 . This forces $d_0 + 1 \leq d_1$, so the second must be at least one day later. This constraint was for when we originally wanted 4 games between each pair. We later found this not feasible and switched to 2 games between each pair, so this constraint does not affect the scheduling.

No Games Closer Than 5 Days for Same Team We want to enforce a minimum of a 5-day gap between any two of t 's games. We use a window W of length 5 days to ensure at max one game is played, represented as:

$$\forall t, \forall W \subseteq \{d, \dots, d+4\} : \sum_{\substack{(i,j,k) \\ i=t \text{ or } j=t}} \sum_{d' \in W} x_{(i,j,k),d'} \leq 1.$$

This will guarantee team t can not play more than once in any 5-day stretch.

Uniform Daily Load We want to spread 140 games across a total of 240 days as evenly as possible. One way would be to force one game on any given day so that exactly 140 distinct days each host one game, leaving 100 idle days. We cannot do better than that in terms of *variance* = 0 among used days. First, let

$$\text{maxLoad} = \left\lceil \frac{|O|}{|D|} \right\rceil = \left\lceil \frac{140}{240} \right\rceil = 1$$

This constraint can be represented as:

$$\forall d \in D : \sum_{(i,j,k) \in O} x_{(i,j,k),d} \leq \text{maxLoad}$$

3.6.4 Objective and Solving

Our only objective is feasibility under the above requirements. All important fairness and spacing criteria are already in the constraints. Hence, we simply need to find any solution that satisfies the listed constraints.

We can solve this binary integer program with CBC. As soon as it finds a feasible assignment, it stops (since there is no objective to improve). The solution and code can be found in the appendix.

3.7 Schedule Effectiveness and Attractiveness

Finally, as we have built our schedule, we must test it and compare it to other competing schedules. We have already built our schedule to optimize fairness as much as possible as well as total travel distance. Therefore, we create metrics for effectivity and attractiveness and compare our schedule with competing styles—namely knockout, round robin, Swiss, and the FIVB schedule.

We will use the FIVB rating model explained in section 2.2.2 to update ratings after every match and obtain win probabilities for each matchup. As a reminder, the FIVB model assumes a normal distribution of game outcomes depending on the difference in rating of two teams. Rating updates also depend on the final score of the victory—for example, a 3-0 win will be awarded more than a 3-2 win against the same rated opponent.

3.7.1 Variable Definitions

Table 9. Variable Definitions

Variable	Definition
E	Effectivity
I, I_{avg}	Intensity, Average Intensity
C, C_{avg}	Closeness, Average Closeness
H, H_{avg}	Entropy, Average Entropy
M	Number of matches (number of pairs (i, j))
S_i	Strength/Score of team i
$p, p_{i>j}$	Probability team i beats team j

3.7.2 Effectivity Metric

Effectivity is meant to measure how close the results of a tournament/schedule is with the “true” ranking of competitors by skill (which we take to be FIVB rankings). For this, we choose to use a **weighted inversion** metric. Let the teams be ordered by FIVB rating. In particular, an **inversion** occurs if team i finishes the league at a place $j > i$. The effect of an inversion is greater when it occurs higher in the rankings. If a schedule is structured so that the 3rd highest ranked team can consistently place 2nd (for example by seeding), this is much more important than if the same thing happens for 9th and 10th place.

Mathematically, the effectivity^[14] can be expressed as

$$E = \sum_{i < j} \sum_{k=i+1}^j \frac{1}{\ln k},$$

where the outer sum is over all places i where an inversion is present. From this we can see a larger displacement (if 5th rank places 1st) will add more than a smaller displacement (5th rank placing 4th). The reciprocal of $\ln k$ is larger for smaller

values of k , which follows from our previous assertion that we value inversions at higher ranks more. Because of this, we prefer schedules with a **lower** effectivity metric.

3.7.3 Attractiveness Metric

We would also like to optimize the **attractiveness** of the schedule. In general, any matchup can be intense or very popular among an audience for a few reasons: the teams themselves are very strong/popular, or a particular the skill of both teams in the matchup are very close. This can influence the **uncertainty** of the outcome of a match, and thus the thrill that draws a large audience and leaves people talking about the match. Because of this, we choose three metrics for attractiveness: **Intensity**, **Closeness**, and **Entropy**.

Intensity We define intensity of a match to be the sum of the strengths of the teams (by FIVB score), or $I = S_i + S_j$ for a matchup (i, j) . Then, the intensity of a tournament is a simple average of intensity over all matches. Given there are M matchups in a schedule, the average intensity is then

$$I_{avg} = \frac{1}{M} \sum_{(i,j)} S_i + S_j.$$

This means some schedules would be punished by too many low-intensity games, while schedules with only high-intensity games will have a higher average. Obviously, though, schedules should be balanced between low-intensity and high-intensity games, so we pair this metric with others.

Closeness For a matchup (i, j) , one way to measure the closeness is $-|S_i - S_j|$, or the negative absolute difference of their FIVB scores. The closer the scores, the closer the match is expected to be, and thus the more entertaining for viewers. We can normalize this metric and average over all matches to get our closeness metric:

$$C_{avg} = \frac{1}{M} \sum_{(i,j)} \left(1 - \frac{|S_i - S_j|}{\max(S_i, S_j)} \right).$$

For each match, a closeness score $C = 1$ indicates a close (and captivating) match, while a score of $C = 0$ indicates a blowout. Therefore, we prefer a larger closeness average for schedules.

Entropy Entropy measures the uncertainty of the outcome predictions of the match by FIVB ratings. For example, in matchup (i, j) we may have that the probability of winning is $(0.6, 0.4)$ or $(0.9, 0.1)$. The latter case has less entropy since there is more certainty that team i will win. Note that these probabilities are obtained from the FIVB ranking method, meaning this entropy is inherently based on where the scaled difference Δ puts the win probabilities in a certain matchup on a normal distribution. We compute the average entropy over all matches with the Shannon Entropy measure, or

$$H_{avg} = -\frac{1}{M} \sum_{(i,j)} (p \log(p) + (1 - p) \log(1 - p)).$$

Where p is the probability team i beats team j . Entropy reflects how confident the FIVB system is in predicting winning teams per matchup, and a larger value indicates more uncertainty. However, a certain level of uncertainty is not necessarily preferred, as it just reflects the real world process being predicted.

3.7.4 Results

First, we tested these metrics of effectivity and attractiveness for different base scheduling systems: knockout, double knockout, single round robin, double round robin, and Swiss. Note that these are averages over many trials, using the initial FIVB strengths for all 20 teams.

This gives us a general idea of which styles perform better at what, and also aligns with some of our researched trends^[11] (like knockout for highest intensity). However, round robin has a high effectivity, which is unexpected since round robin is reportedly most efficient from research (though possibly by different metrics).

Next, we simulated our 140 game preliminary season to update ratings, and then observed which of Swiss, round robin, or knockout would produce the best results by these metrics. One would expect the results to be very similar to the table above, but since the FIVB scores have been altered (possibly by a significant amount), it is very possible different styles are preferred. We ran 1000 simulations of each, and the histograms in Figures 7 and 8 shows the final results.

The final results for round robin and our code for these simulations are in the appendix. For all three, closeness seems to follow a normal distribution centered at 0.5, which is sensible that difference in skill levels end up averaging out. Entropy

Style	Effectiveness	Closeness	Intensity	Entropy
Knockout	3.008	0.688	541.341	0.494
Double Knockout	4.246	0.673	552.560	0.496
Single Round Robin	9.461	0.546	481.991	0.399
Double Round Robin	10.070	0.546	481.991	0.399
Swiss	5.454	0.694	490.108	0.526

Table 8: Comparison of tournament styles based on effectiveness, closeness, intensity, and entropy.

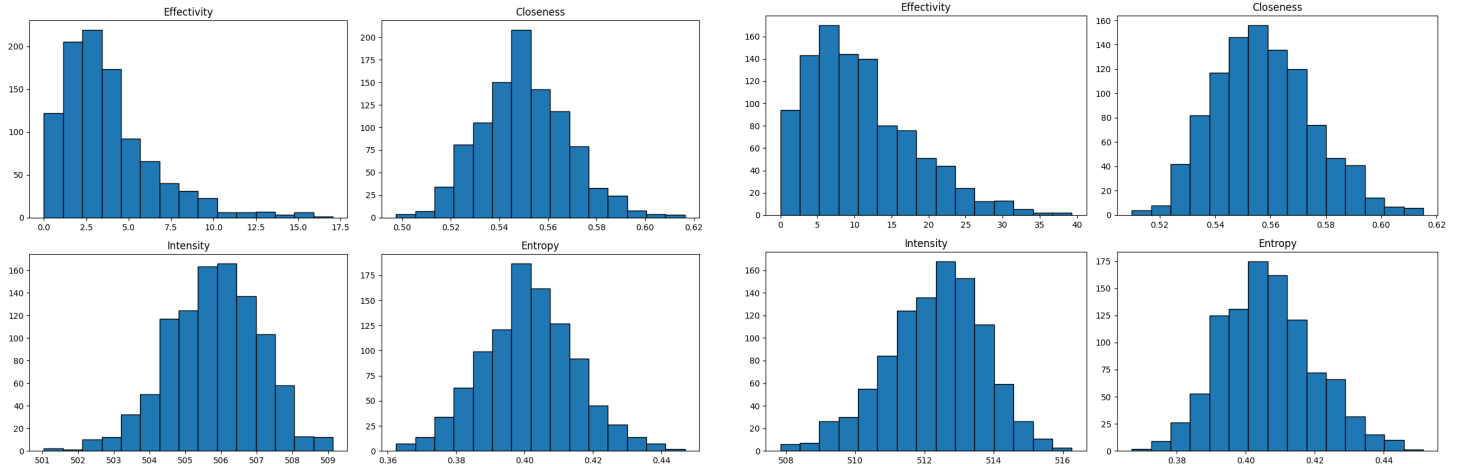


Figure 7: (Double) Knockout

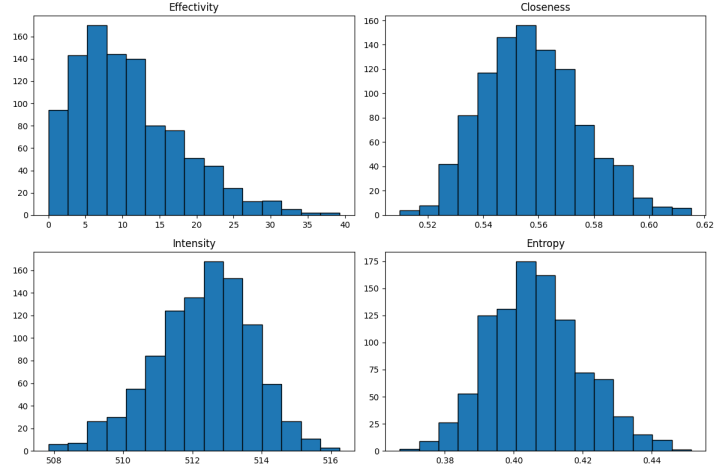


Figure 8: Swiss

also seems similar for all three, which is a good indicator that the model's uncertainty matches the real-world uncertainty of the outcome of the matches.

The metrics that are significantly different are effectivity and intensity. In order of least to greatest effectivity, we have knockout < Swiss < round robin; for intensity, we have knockout < Swiss < round robin. Note that lower effectivity is preferred and higher intensity is preferred. Going for a balance between these, one might choose Swiss, but we choose to stay with the Knockout structure as we value effectivity of the tournament more than a few more intensity points (since all remain around 500-520).

Therefore, we have decided on a final schedule for all women's volleyball teams in the GSL. In the preliminary round, each team will face 7 other teams twice over the course of 8 months. Then, the top 8 rated teams (by FIVB ratings) will advance to the championship round, which is a double knockout bracket (essentially Knockout with one loss allowed before elimination, or a loser's bracket). Single knockout cannot provide a definitive ranking for 8 teams, so we decided to use double knockout to make measuring effectivity easier. The championship will last up to a month to fit within the GSL's schedule, and will be held at different countries every year similar to the Olympics or VNL.

Our schedule fully addresses the IMMC's goals of fairness, sustainability, and geographic representation. Our team selection ensures worldwide geographic representation. We have carefully considered sustainability in balancing the total distance traveled by all teams with ensuring teams can sufficiently change their rating by playing a limited set of other teams. Finally, we have optimized the schedule for fairness to ensure no team experiences out of proportions home/away advantages and that every team has sufficient time between matches for both travel and rest.

We are prompted to determine how many games each team must play to determine a winner of the league. Each team must play 14 games in the preliminary round, and a minimum of 7 rounds must occur in the championship (essentially a single knockout), of which the winning team plays 3 rounds. Therefore, each team should expect to play between 14 and 17 matches to determine a league winner.

4 Expand the Model (Part 3)

4.1 New GSL Teams (3a)

The GSL desires to add four more teams into their league; however, we are unable to follow our exact criteria as stated in 2.3. While criteria 1 through 4 are followed, we must make adjustments to criterion 5 to account for a few exceptions. See the adjusted criterion below:

1. There must be four or five teams per continent.

Justification: We adjust this criteria from 2.3 in order to prioritize geographic diversity while also allowing more high performing teams like Germany to have a spot in the GSL. Note that Oceania is an exception again (with only 2 teams) as only Australia represents Oceania, and we do not want Australia to have four teams representing Oceania.

Table 9: 4 New Teams for the Global Sports League

Rank	Team	Continent	Venue	FIVB Points
12	Germany	Europe	Max-Schmeling-Halle	222.03
18	Cuba	North America	Ciudad Deportiva Coliseum	145.17
21	Egypt	Africa	Cairo Stadium Indoor Halls Complex	54.62
24	Algeria	Africa	Salle Harcha Hassen	28.00

Note: The full GSL list of teams with coordinates is visible in Appendix 9.3.1.

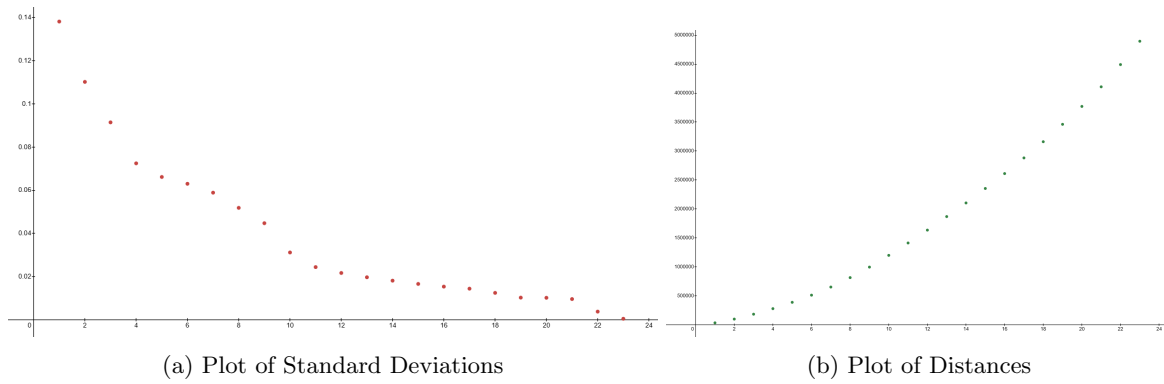
4.2 Finding the Optimal k-value for 24 Teams

We will apply the same approach as we did for 20 teams and run through all the values of k from $k = 1$ to $k = 23$, find the transition matrix, find the steady state, and find the standard deviation of the entries in the steady state. The result is shown in the table below.

Table 10: Standard Deviation and Directed Distance Sum for Different k values

k	std()	Distance	k	std()	Distance	k	std()	Distance
1	0.13819	33385	9	0.04478	995982	17	0.01446	2881559
2	0.11024	98135	10	0.03123	1198991	18	0.01251	3164205
3	0.09149	182883	11	0.02445	1412614	19	0.01031	3461709
4	0.07251	278358	12	0.02173	1634986	20	0.01023	3770926
5	0.06620	387907	13	0.01974	1868582	21	0.00965	4109316
6	0.06304	511665	14	0.01817	2104018	22	0.00383	4493534
7	0.05893	651721	15	0.01666	2353731	23	0.00052	4896595
8	0.05190	814248	16	0.01541	2611214			

To balance minimizing standard deviation while also minimizing distance, we will choose $k = 11$. We see the effects of diminishing returns around $k = 11$. From $k = 1$ to $k = 10$, each additional neighbor decreases approximately $\approx 0.010=0.014$ in standard deviation. The transition from $k = 11 \rightarrow 12$ yields only a further $\Delta\text{std} = 0.0027$, which is just 8% of the first-step gain. Beyond $k = 11$, every additional edge reduces standard deviation by less than 0.002 while adding over 220,000 km of travel. The total directed distance at $k = 11$ is 1.41 million km, which is about 71% of what it would be at $k = 14$ (2.10 million km) and only 32% of the endpoint at $k = 23$. Relative to $k = 10$, you pay an extra $\sim 213,000$ km ($\approx 18\%$), but reap a 22% reduction in standard deviation. For more technicality, we could perform a weighted-normalization check from which we see that $k = 11$ is ideal.

Figure 9: Steady state analysis dependent on k

We find the transition matrix for $k = 11$ and proceed to create the schedule. The transition matrix for $k = 11$ can be found in the appendix. Similarly to earlier, we will have the same setup and constraints for the MILP. However, for the daily load cap, that becomes 2 instead of 1, as

$$\text{maxLoad} = \left\lceil \frac{|O|}{|D|} \right\rceil = \left\lceil \frac{264}{240} \right\rceil = 2.$$

The optimal schedule can be found in the appendix.

4.3 New GSL Schedule

We perform the same analysis on effectivity, intensity, closeness, and entropy to decide the format of the championship for the 24 team league. Again, knockout is the best for effectivity. An argument can be made for round robin, since it is both more intense and better for closeness (with a mean closer to 0.5 than knockout), but we will use a knockout format for this schedule as well. The Swiss histogram can be found in the appendix. The extension to 24 teams has affected our model and

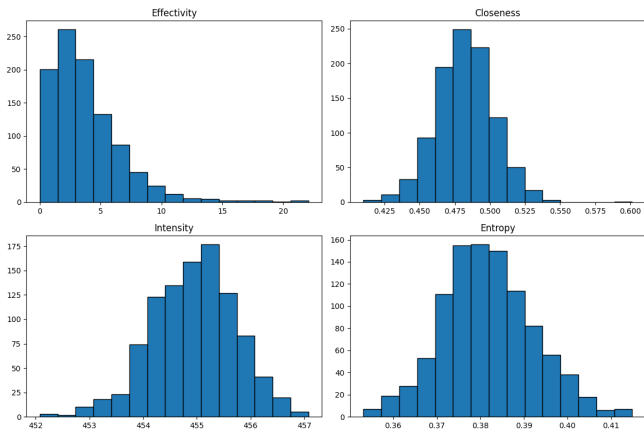


Figure 10: (Double) Knockout for 24 team schedule

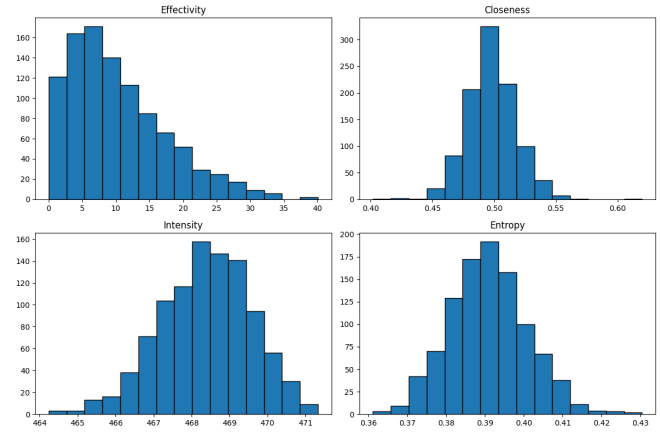


Figure 11: Round Robin for 24 team schedule

schedule in multiple ways. Though we worked to optimize travel distance constraints with valid preliminary scheduling, the total travel distance increased by around 800,000 km (more than double). This is due not only to the increased number of teams but the fact that we chose for each team to play $k = 11$ teams as opposed to $k = 7$ to ensure the graph was connected enough. If we were to be more environmentally cautious, we could have chosen $k = 9$ to halve that increase to about 400,000. But, as we have seen from the distance tables for different k values, increasing teams causes a quadratic or even exponential increase in total distance, so we should be very careful when creating schedules.

The GSL has maintained geographic representation as explained in our criteria for choosing the 4 extra teams. With the exception of Oceania, every continent has at least 4 teams representing them. Additionally, though implementing our MILP model was more difficult (fitting more games into the same time period), we were still able to uphold the IMMC's criteria of fairness.

Since we have chosen $k = 11$ alongside the increase in teams, every team must play 22 games in the preliminary season. Since we have stuck with the choice of a Knockout championship, each team in the championship plays between 1-3 additional

games. Therefore, each team must play between 22 and 25 games now to determine a league winner.

We also decided to measure the FIVB Women’s World Championship 2025 schedule^[15] by these metrics to see how it compares. The FIVB schedule first includes a pooling round, where 32 teams are split into groups of 4 and have matches against all others in their pool. Then, the top two teams from each pool advances to a 16 player knockout tournament. The full histogram for the FIVB 2025 schedule can be found in the appendix, but the main takeaways we found is that their schedule has more intense games and their closeness index is on average greater than ours. This is likely due to the World Championship including all of the highest-rated teams, but not necessarily geographically diverse. We found that our schedule has a better effectivity score on average, and the selection of the groups of 4 in the pooling round are not geographically optimal to minimize travel distance for teams. Our schedule intentionally accounts for this, and we believe it is something the FIVB schedule could improve on.

5 Generalizing Our Model (Part 4)

5.1 Generalizing Rating

If we were to generalize this model for a different sport, the essence of the FIVB rating system could still be used. The basis of the model—a dynamic point system in which teams gain and lose points depending on a predetermined probability that they win the game based on their current rating—is a popular one, and it is used in many competitive contexts, such as the Elo rating system in chess. Researchers have found it so useful that Elo-like systems are also used in evaluating large language models.^[16]

The only aspect of the model that would certainly need changing is the index, which currently exists from 1 to 6 to represent different possible outcomes for a game. When applying this model to a sport with a different score system within each game, we would need to change the index to match the normalized scores. We can separate other sports into two types of scoring systems: finite score margins and infinite score margins.

Finite score margins are sports in which there is a “first to x ” rule for determining the winner. For volleyball, it is first to 3; tennis is similar. These sports are easiest to adapt our rating model to because we can simply adapt the index to reflect the possible score outcomes.

Infinite score margins are sports where teams can theoretically score an infinite number of points—soccer is one such example. We can adapt our model to these sports by assigning indices not to specific outcomes but to score differentials. This is essentially the same as what we currently use for volleyball: 3-0 is winning by 3, 3-1 is winning by 2, etc. Assigning a finite number of indices is appropriate even when there are a potentially infinite number of score differentials because the larger the differential, the less likely it is to occur.

5.2 Generalizing Matchups

When creating the matchups in our schedule, we emphasized geographic diversity and environmental sustainability. When generalizing our model to other team sports, one may have to be cautious with geographic diversity, since we ran into some issues with the only team from Oceania being Australia in the FIVB. For sports where there is not already an international presence, teams would have to be selected from national or regional competitions, if they exist. If not, the sport may be too young to be represented by the GSL.

Additionally, the emergence of e-sports completely changes our translation of “environmental sustainability” to “total travel distance” for volleyball. Some sports may require facilities that harm the environment to create, as we discussed when deciding a sport, and this is also the case for e-sports with the environmental impact of high powered computing. When generalizing this model, we certainly need a more general indicator of environmental sustainability.

We feel that the GSL’s definition of **team sport** could be broadened, especially taking into account the different team dynamics present in many sports. Namely, it is interesting that this definition (minimum of five players per team actively participating) excludes sports like baseball, where (minimally) the only player actively participating for one team is the batter. Certain team dynamics in certain sports could qualify it as a team sport, in our opinion. This does not exactly impact our model, since if we chose baseball for the sport, it would still include travel of the entire team to different venues.

5.3 Generalizing Schedule Creation

Some sports or team activities are not necessarily team versus team. Our model exclusively considers team vs. team matchups, though we could extend the MILP model to consider sports where many teams compete at once.

We can also generalize our metrics of effectivity and attractiveness to different sports styles. Conceptually they are useful, as measuring audience interest in games and how well the results of a schedule reflect real skill ranking can be used for schedule building regardless of the sport. However, we define these metrics using FIVB ratings, which are limited as we discussed previously. Extending these notions to a broader rating system for sports would be a good idea. Additionally,

we could consider the possibility of prize allocation in tournaments, and the resulting concern of strategy-proofing how we allocate prizes.

6 Strengths and Weaknesses

6.1 Strengths

Our rating system is well-suited to our model because it allows for dynamic updating of ratings that takes into consideration the changing current ratings of teams, which is essential for the diffusion system our model relies on.

Our MILP model is particularly strong in that by formulating all the requirements into linear equalities or inequalities, the model ensures 100% feasibility with respect to the problem specification. No post-hoc repair or heuristic patching is needed. If the model returns “feasible,” all constraints are satisfied exactly. Additionally, the model has modularity and extendability. The same template can be readily applied to different degrees (whether it be a 7-regular or 11-regular graph) simply by changing the adjacency matrix or the parameter k . New constraints can be added as additional linear terms or constraints without rewriting the core model at all. The MILP for generating a k -regular graph of minimal travel distance guarantees a provably optimal network, which eliminates bias that nearest-neighbor or greedy graph algorithms can introduce. By coupling this with the scheduling MILP, we can jointly explore how different connectivity patterns affect schedule feasibility and equity. Tests on 24-team, 264-game instances solve in a few minutes on a standard desktop, making the approach practical for real leagues.

Another strength of our model is that we have built a diverse set of metrics to evaluate the effectivity and attractiveness of a schedule, giving deep insight into the dynamics of a tournament beyond its results. It is also widely applicable, as we have shown by using the FIVB ranking system and simulating our schedule, general knockout/swiss/round robin schedules, and the 2025 Women’s World Championship schedule.

6.2 Weaknesses

One weakness for the MILP is its scalability. For the scheduling MILP, the number of binary variables is $|O| \times |D|$ (e.g., 264 games $\times$ 240 days $\approx$ 63,360 binaries). Adding more teams or condensing into fewer days causes a quadratic or exponential explosion, quickly exceeding solver capacity.

The k -regular graph model has $\frac{n(n-1)}{2}$ edge variables; for $n > 50$, this alone becomes impractical without further decomposition or symmetry-breaking cuts. The k -regular graph MILP also contains a high degree of symmetry (any permutation of node labels yields an equivalent solution), which can slow CBC by exploring symmetric subtrees. Without specialized symmetry-breaking constraints, solver time may spike unpredictably.

We also oversimplified some parts of this model. The distance metric for the k -regular graph uses straight-line geodesic kilometers, ignoring real-world travel networks, visa restrictions, or time-zone effects. Thus, the “optimal” graph may not be operationally optimal. Similarly, scheduling assumes all days are interchangeable and ignores blackout dates (holidays), venue availability windows, or TV broadcast time slots.

Our entire model is quite split up, in some sense. That is, to create our schedule we used a MILP algorithm paired with Markov Chain analysis, then another MILP algorithm, and finally analysis of simulation results for some metrics. It is not exactly easily replicable as a whole model, though it reflects us trying to take into account many different factors at once requiring some of our own input midway to do so.

7 Conclusion

Our primal desire for competition and physical prestige has glorified sports in every age of time, from gladiators to the Olympics. Globalization has increased the caliber of sports we can observe, and consequently the level we strive towards as athletes. It is imperative, then, that all competitions are fair and that the outcomes reflect the skill level of competitors. That is exactly what we have done in this paper: we have created not only a feasible, effective schedule, but also one that takes into account environmental sustainability and geographic diversity in the age of globalization. Our schedule ensures that all competitors play a fair game and that all audience members get a bang for their buck as they watch the most exciting games that humanity has to offer.

8 Works Cited

1. Mooney, T. (2024, August 2). Paris 2024 Olympics: What makes an Olympic sport? Olympics.com; International Olympic Committee. <https://www.olympics.com/en/news/paris-2024-olympics-what-makes-an-olympic-sport>
2. 120 years of Olympic history: athletes and results. (2018). Kaggle. <https://www.kaggle.com/datasets/heesoo37/120-years-of-olympic-history-athletes-and-results>
3. Colley, W. (2025). Properly Weighted Olympic Medal Counts. Colley Rankings. <https://www.colleyrankings.com/olympics2024/explanation.html>
4. What are the sources of income for sports leagues? (n.d.). Statscore; Statscore. <https://www.statscore.com/market-research/what-are-the-sources-of-income-for-sports-leagues/>
5. Examining how digital consumers around the world engage with sports, and the opportunities for sports brands and marketers Sports Around the World. (2019). Global Web Index. <https://iccopr.com/wp-content/uploads/2019/03/Sports-Around-the-World-report.pdf>
6. Water Polo Pool Guide - Dimensions & Markings. (2022, February 28). Aqua Blue Pools. <https://www.aquabluepools.net/blog/swimming-pools/water-polo-pool-guide-dimensions-markings/>
7. World Aquatics Communication Department. (2024, August 6). A historic moment as World Aquatics commits to the UN Sports for Climate Action Framework. World Aquatics; World Aquatics. <https://www.worldaquatics.com/news/4077093/a-historic-moment-as-world-aquatics-commits-to-the-un-sports-for-climate-action-framework>
8. Poole, C. (2025, January 15). EFL Launches Revamped Scheme For Soccer Sustainability. Forbes; Forbes Media LLC. <https://www.forbes.com/sites/clairepooleesp/2025/01/15/efl-launches-revamped-scheme-for-soccer-sustainability/>
9. The Official FIVB women's Volleyball World Ranking. (2025, January 8). Volleyball World; Volleyball World. <https://en.volleyballworld.com/volleyball/world-ranking/women>
10. Rankings. (n.d.). Fédération Internationale de Volleyball; FIVB. <https://www.fivb.com/volleyball/rankings/>
11. Devriesere, K., Csató, L., & Goossens, D. (2025). Tournament design: A review from an operational research perspective. *European Journal of Operational Research*, 324(1), 1–21. <https://doi.org/10.1016/j.ejor.2024.10.044>
12. Wessel, S., Hurkens, C., & Spieksma, F. (2025). Fair Schedules for Single Round Robin Tournaments with Ranked Participants. *ArXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2502.04159>
13. Goossens, D. R., & Spieksma, F. C. R. (2012). The Carryover Effect Does Not Influence Football Results. *Journal of Sports Economics*, 13(3), 288–305. <https://doi.org/10.1177/1527002511402932>
14. Sziklai, B. R., Biró, P., & Csató, L. (2022). The efficacy of tournament designs. *Computers & Operations Research*, 144, 105821-. <https://doi.org/10.1016/j.cor.2022.105821>
15. Match Schedule Released for FIVB Volleyball Women's World Championship 2025 in Thailand. (2025, March 11). Fédération Internationale de Volleyball; FIVB. <https://www.fivb.com/match-schedule-released-for-fivb-volleyball-womens-world-championship-2025-in-thailand/>
16. Elos and Benchmarking LLMs. (2024, September 24). Quantable Analytics; Quantable Analytics. <https://www.quantable.com/analytics/elos-and-benchmarking-llms/>

9 Appendix

9.1 Problem 1

Sport Equity Python Code (1a)

```

1 import pandas as pd
2 import numpy as np
3
4 medals = pd.read_csv('data/medals.csv')
5
6 bigMedals = pd.read_csv('data/athlete_events.csv')
7 dateProcess = bigMedals[bigMedals['Year'] > 1980]
8 #Basketball, Football, Ice Hockey, Handball, Water Polo, Hockey, Volleyball, Softball, Baseball, Rugby
   Sevens
9
10 def process_sport_medals(dateProcess, sport_name):
11     sport_winners = dateProcess[(dateProcess['Sport'] == sport_name) & (~pd.isna(dateProcess['Medal']))]
12
13     sport_dict = {}
14     for index, row in sport_winners.iterrows():
15         key = row['NOC'] + row['Medal']
16         if key not in sport_dict:
17             sport_dict[key] = 0
18         sport_dict[key] += 1
19
20     return sorted(sport_dict.items(), key=lambda item: item[1], reverse=True)
21
22 def sport_point(dateProcess, sport_name):
23     sport_winners = dateProcess[(dateProcess['Sport'] == sport_name) & (~pd.isna(dateProcess['Medal']))]
24     goldPoints = 6
25     silverPoints = 3
26     bronzePoints = 2
27
28     sport_dict = {}
29     for index, row in sport_winners.iterrows():
30         key = row['NOC']
31         if key not in sport_dict:
32             sport_dict[key] = 0
33         if row['Medal'] == 'Gold':
34             sport_dict[key] += goldPoints
35         if row['Medal'] == 'Silver':
36             sport_dict[key] += silverPoints
37         if row['Medal'] == 'Bronze':
38             sport_dict[key] += bronzePoints
39
40     return sorted(sport_dict.items(), key=lambda item: item[1], reverse=True)
41
42 # Unique team sports featured in the Olympics
43 sports = [
44     "Basketball", "Football", "Ice Hockey", "Handball",
45     "Water Polo", "Hockey", "Volleyball", "Softball",
46     "Baseball", "Rugby Sevens"
47 ]
48
49 for sport in sports:
50     print(f"Results for {sport}:")
51     results = process_sport_medals(dateProcess, sport)
52     print(results)
53
54 standardDeviations = []
55 for sport in sports:
56     print(f"Results for {sport}:")
57     results = sport_point(dateProcess, sport)
58     data = []
59     for item in results:
60         data.append(item[1])
61     print(results)
62     standardDeviations.append([sport, np.std(data)/(len(results)**2)])
63 print(standardDeviations)

```

9.2 Problem 2

Table 11: 20 Selected Teams for the Global Sports League with Coordinates

Rank	Team	Continent	Venue	Coordinates
1	Italy	Europe	PalaLottomatica	41.8253° N, 12.4666° E
2	Brazil	South America	Arena da Baixada	25.4481° S, 49.2769° W
3	USA	North America	American Sports Center	33.8102° N, 117.9055° W
4	Türkiye	Asia	Sinan Erdem Dome	40.9884° N, 28.8539° E
5	China	Asia	Beijing National Stadium	39.992935° N, 116.396446° E
6	Poland	Europe	Atlas Arena	51.75694° N, 19.42500° E
7	Japan	Asia	Ariake Arena	35.6406° N, 139.7971° E
8	Canada	North America	Rogers Centre	43.641796° N, 79.390083° W
9	Netherlands	Europe	Johan Cruyff Arena	52.314224° N, 4.941841° E
10	Serbia	Europe	Tašmajdan Sports and Recreation Center	44.80934° N, 20.47067° E
11	Dominican Republic	North America	Centro Olímpico Juan Pablo Duarte	18.4774° N, 69.917° W
12	Thailand	Asia	Indoor Stadium Huamark	13.7563° N, 100.5018° E
13	Puerto Rico	North America	Coliseo de Puerto Rico José Miguel Agrelot	18.4277° N, 66.0614° W
14	Argentina	South America	Estadio Luna Park	34.602032° S, 58.369022° W
15	Colombia	South America	Estadio Metropolitano Roberto Meléndez	10.9269° N, 74.8006° W
16	Kenya	Africa	Kasarani Indoor Arena	1.2195° S, 36.8867° E
17	Peru	South America	Estadio Nacional del Perú	12.067278° S, 77.033722° W
18	Cameroon	Africa	Yaoundé Multipurpose Sports Complex	3.8742° N, 11.5119° E
19	Australia	Oceania	Accor Stadium	33.847057° S, 151.063156° E
20	Australia	Oceania	Accor Stadium	33.847057° S, 151.063156° E

Transition Matrix Finder When k=4 for 20 Teams

```

1 import pandas as pd
2 import numpy as np
3 from geopy.distance import geodesic

```

```

4 import pulp
5
6 data = [
7     {"country": "italy", "name": "PalaLottomatica", "lat": 41.8253, "lon": 12.4666},
8     {"country": "poland", "name": "Atlas Arena", "lat": 51.75694, "lon": 19.42500},
9     {"country": "netherlands", "name": "Johan Cruyff Arena", "lat": 52.314224, "lon": 4.941841},
10    {"country": "serbia", "name": "Ta majdan Sports and Recreation Center", "lat": 44.80934, "lon":
11    20.47067},
12    {"country": "usa", "name": "American Sports Center", "lat": 33.8102, "lon": -117.9055},
13    {"country": "canada", "name": "Rogers Centre", "lat": 43.641796, "lon": -79.390083},
14    {"country": "dominican republic", "name": "Centro Ol mpico Juan Pablo Duarte", "lat": 18.4774, "lon":
15    -69.9170},
16    {"country": "puerto rico", "name": "Coliseo de Puerto Rico Jos Miguel Agrelot", "lat": 18.4277, "lon":
17    -66.0614},
18    {"country": "brazil", "name": "Arena da Baixada (Ligga Arena)", "lat": -25.4481, "lon": -49.2769},
19    {"country": "argentina", "name": "Estadio Luna Park", "lat": -34.602032, "lon": -58.369022},
20    {"country": "colombia", "name": "Estadio Metropolitano Roberto Mel ndez", "lat": 10.9269, "lon":
21    -74.8006},
22    {"country": "peru", "name": "Estadio Nacional del Per ", "lat": -12.067278, "lon": -77.033722},
23    {"country": "australia", "name": "Accor Stadium (Stadium Australia)", "lat": -33.847057, "lon":
24    151.063156},
25    {"country": "australia2", "name": "Accor Stadium (Stadium Australia)", "lat": -33.847057, "lon":
26    151.063156},
27    {"country": "t rkiye", "name": "Sinan Erdem Dome", "lat": 40.9884, "lon": 28.8539},
28    {"country": "china", "name": "Beijing National Stadium ( Birds Nest)", "lat": 39.992935, "lon":
29    116.396446},
30    {"country": "japan", "name": "Ariake Arena", "lat": 35.6406, "lon": 139.7971},
31    {"country": "thailand", "name": "Indoor Stadium Huamark", "lat": 13.7563, "lon": 100.5018},
32    {"country": "kenya", "name": "Kasarani Indoor Arena (Moi International Sports Centre)", "lat":
33    -1.2195, "lon": 36.8867},
34    {"country": "cameroon", "name": "Yaound Multipurpose Sports Complex", "lat": 3.8742, "lon": 11.5119}
35 ]
36 df = pd.DataFrame(data)
37 n = len(df)
38 k = 4
39
40 dist = {}
41 for i in range(n):
42     dist[i] = {}
43     coord_i = (df.loc[i, "lat"], df.loc[i, "lon"])
44     for j in range(n):
45         coord_j = (df.loc[j, "lat"], df.loc[j, "lon"])
46         dist[i][j] = round(geodesic(coord_i, coord_j).km, 2)
47
48 model = pulp.LpProblem("k-regular_graph", pulp.LpMinimize)
49
50 x = {(i, j): pulp.LpVariable(f"x_{i}_{j}", cat="Binary") for i in range(n) for j in range(i+1, n)}
51
52 model += pulp.lpSum(dist[i][j] * x[(i, j)] for i in range(n) for j in range(i+1, n))
53
54 for i in range(n):
55     model += pulp.lpSum(x[(min(i, j), max(i, j))] for j in range(n) if j != i) == k
56
57 model.solve(pulp.PULP_CBC_CMD(msg=False))
58
59 adj = np.zeros((n, n), dtype=int)
60 for (i, j), var in x.items():
61     if pulp.value(var) > 0.5:
62         adj[i, j] = adj[j, i] = 1
63
64 adj_df = pd.DataFrame(adj, index=df['name'], columns=df['name'])
65 adj_df.to_csv('transition_matrix.csv', index=True)
66
67 print("Transition matrix saved as 'transition_matrix.csv'")
68 print(adj_df)

```

STD and Distance Minimizer for 20 Teams

```

1 import pandas as pd
2 import numpy as np
3 from geopy.distance import geodesic
4

```

0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
1	0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0
1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	1	1	0	0	1	0	0	0	0	0	0
0	0	0	0	1	0	1	0	0	1	1	0	0	0	0	0	0	0	0	1
0	0	0	0	1	1	0	0	0	0	0	1	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	1	0	1	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	1	0	1	1	0	0	0	0	0
0	0	0	0	1	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	0	0	0	1	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	1	0	0
0	0	0	0	1	0	1	1	1	0	0	0	1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	1
0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	1	1	0	0
0	0	0	1	0	0	0	0	0	0	1	0	1	0	0	1	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1
1	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	1	0

Figure 12: The transition matrix for k=4 for 20 Teams

```

5 data = [
6     {"name": "PalaLottomatica", "lat": 41.8253, "lon": 12.4666},
7     {"name": "Atlas Arena", "lat": 51.75694, "lon": 19.42500},
8     {"name": "Johan Cruyff Arena", "lat": 52.314224, "lon": 4.941841},
9     {"name": "Ta majdan Sports Center", "lat": 44.80934, "lon": 20.47067},
10    {"name": "American Sports Center", "lat": 33.8102, "lon": -117.9055},
11    {"name": "Rogers Centre", "lat": 43.641796, "lon": -79.390083},
12    {"name": "Centro Ol. Juan P. Duarte", "lat": 18.4774, "lon": -69.9170},
13    {"name": "Coliseo PR Agrelot", "lat": 18.4277, "lon": -66.0614},
14    {"name": "Arena da Baixada", "lat": -25.4481, "lon": -49.2769},
15    {"name": "Estadio Luna Park", "lat": -34.602032, "lon": -58.369022},
16    {"name": "Estadio Metropolitano", "lat": 10.9269, "lon": -74.8006},
17    {"name": "Estadio Nac Peru", "lat": -12.067278, "lon": -77.033722},
18    {"name": "Accor Stadium AU1", "lat": -33.847057, "lon": 151.063156},
19    {"name": "Accor Stadium AU2", "lat": -33.847057, "lon": 151.063156},
20    {"name": "Sinan Erdem Dome", "lat": 40.9884, "lon": 28.8539},
21    {"name": "Beijing Natl Stad", "lat": 39.992935, "lon": 116.396446},
22    {"name": "Ariake Arena", "lat": 35.6406, "lon": 139.7971},
23    {"name": "Indoor Stad Huamark", "lat": 13.7563, "lon": 100.5018},
24    {"name": "Kasarani Indoor", "lat": -1.2195, "lon": 36.8867},
25    {"name": "Yaounde Complex", "lat": 3.8742, "lon": 11.5119}
26 ]
27 df = pd.DataFrame(data)
28 n = len(df)
29
30 dist = np.zeros((n, n))
31 for i in range(n):
32     for j in range(n):
33         dist[i, j] = geodesic((df.loc[i, "lat"], df.loc[i, "lon"]),
34                               (df.loc[j, "lat"], df.loc[j, "lon"])).km
35
36 std_devs = []
37 tot_dists = []
38
39 for k in range(1, n):

```

```

40 adj = np.zeros((n, n), int)
41 total_distance = 0.0
42
43
44 for i in range(n):
45     nn = np.argsort(dist[i])[1:k+1]
46     for j in nn:
47         adj[i, j] = 1
48         total_distance += dist[i, j]
49
50
51 P = adj.astype(float)
52 P = P / P.sum(axis=1, keepdims=True)
53
54
55 eigvals, eigvecs = np.linalg.eig(P.T)
56 idx1 = np.argmin(np.abs(eigvals - 1))
57 steady = np.real(eigvecs[:, idx1])
58 steady = steady / steady.sum()
59
60 std_devs.append(np.std(steady))
61 tot_dists.append(total_distance)
62
63
64 print(" k | std() | directed-distance-sum (km)")
65 print("-----")
66 for k in range(1, n):
67     print(f"{k:2d} | {std_devs[k-1]:8.5f} | {tot_dists[k-1]:10.0f}")

```

Transition Matrix Finder When k=7

```

1 import pandas as pd
2 import numpy as np
3 from geopy.distance import geodesic
4 import pulp
5
6 data = [
7     {"country": "italy", "name": "PalaLottomatica", "lat": 41.8253, "lon": 12.4666},
8     {"country": "poland", "name": "Atlas Arena", "lat": 51.75694, "lon": 19.42500},
9     {"country": "netherlands", "name": "Johan Cruyff Arena", "lat": 52.314224, "lon": 4.941841},
10    {"country": "serbia", "name": "Ta majdan Sports and Recreation Center", "lat": 44.80934, "lon":
11    20.47067},
12    {"country": "usa", "name": "American Sports Center", "lat": 33.8102, "lon": -117.9055},
13    {"country": "canada", "name": "Rogers Centre", "lat": 43.641796, "lon": -79.390083},
14    {"country": "dominican republic", "name": "Centro Ol mpico Juan Pablo Duarte", "lat": 18.4774, "lon":
15    -69.9170},
16    {"country": "puerto rico", "name": "Coliseo de Puerto Rico Jos Miguel Agrelot", "lat": 18.4277, "lon":
17    -66.0614},
18    {"country": "brazil", "name": "Arena da Baixada (Ligga Arena)", "lat": -25.4481, "lon": -49.2769},
19    {"country": "argentina", "name": "Estadio Luna Park", "lat": -34.602032, "lon": -58.369022},
20    {"country": "colombia", "name": "Estadio Metropolitano Roberto Mel ndez", "lat": 10.9269, "lon":
21    -74.8006},
22    {"country": "peru", "name": "Estadio Nacional del Per ", "lat": -12.067278, "lon": -77.033722},
23    {"country": "australia", "name": "Accor Stadium (Stadium Australia)", "lat": -33.847057, "lon":
24    151.063156},
25    {"country": "australia2", "name": "Accor Stadium (Stadium Australia)", "lat": -33.847057, "lon":
26    151.063156},
27    {"country": "t rkiye", "name": "Sinan Erdem Dome", "lat": 40.9884, "lon": 28.8539},
28    {"country": "china", "name": "Beijing National Stadium (Birds Nest)", "lat": 39.992935, "lon":
29    116.396446},
30    {"country": "japan", "name": "Ariake Arena", "lat": 35.6406, "lon": 139.7971},
31    {"country": "thailand", "name": "Indoor Stadium Huamark", "lat": 13.7563, "lon": 100.5018},
32    {"country": "kenya", "name": "Kasarani Indoor Arena (Moi International Sports Centre)", "lat":
33    -1.2195, "lon": 36.8867},
34    {"country": "cameroon", "name": "Yaound Multipurpose Sports Complex", "lat": 3.8742, "lon": 11.5119}
35 ]
36 df = pd.DataFrame(data)
37 n = len(df)
38 k = 7
39
40 dist = {}
41 for i in range(n):

```

0	1	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	1
1	0	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	1
1	1	0	1	0	1	0	0	0	0	0	0	0	0	1	1	0	0	0	1
1	1	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	1
0	0	0	0	0	1	1	1	0	0	1	1	0	1	0	0	1	0	0	0
0	0	1	0	1	0	1	1	0	0	1	1	0	0	0	0	1	0	0	0
0	0	0	0	1	1	0	1	1	1	1	1	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	1	1	1	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	1	1	0	1	1	1	1	0	0	0	0	0	0	1
0	0	0	0	0	0	1	1	1	0	1	1	1	1	0	0	0	0	0	0
0	0	0	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	1	0	0	0	1	0	1	1	1	1	0
0	0	0	0	1	0	0	0	0	1	0	0	1	0	0	1	1	1	1	0
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	1
0	1	1	1	0	0	0	0	0	0	0	0	1	1	1	0	1	1	1	0
0	1	0	0	1	1	0	0	0	0	0	0	1	1	0	1	0	1	1	0
1	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	0	1	0
1	0	0	1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	0	1
1	1	1	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	1	0

Figure 13: The transition matrix for k=7

```

34 dist[i] = {}
35 coord_i = (df.loc[i, "lat"], df.loc[i, "lon"])
36 for j in range(n):
37     coord_j = (df.loc[j, "lat"], df.loc[j, "lon"])
38     dist[i][j] = round(geodesic(coord_i, coord_j).km, 2)
39
40 model = pulp.LpProblem("k-regular_graph", pulp.LpMinimize)
41
42 x = {(i, j): pulp.LpVariable(f"x_{i}_{j}", cat="Binary") for i in range(n) for j in range(i+1, n)}
43
44 model += pulp.lpSum(dist[i][j] * x[(i, j)] for i in range(n) for j in range(i+1, n))
45
46 for i in range(n):
47     model += pulp.lpSum(x[(min(i, j), max(i, j))] for j in range(n) if j != i == k)
48
49 model.solve(pulp.PULP_CBC_CMD(msg=False))
50
51 adj = np.zeros((n, n), dtype=int)
52 for (i, j), var in x.items():
53     if pulp.value(var) > 0.5:
54         adj[i, j] = adj[j, i] = 1
55
56 adj_df = pd.DataFrame(adj, index=df['name'], columns=df['name'])
57 adj_df.to_csv('transition_matrix.csv', index=True)
58
59 print("Transition matrix saved as 'transition_matrix.csv'")
60 print(adj_df)

```

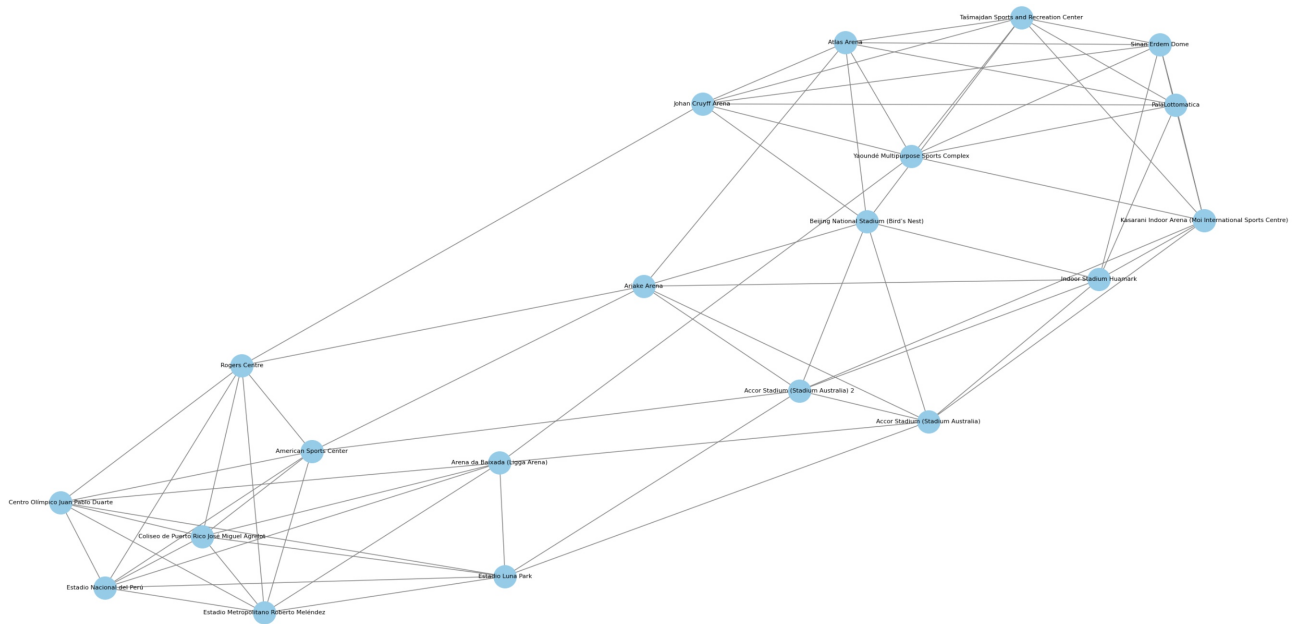


Figure 14: Visualized Transition Matrix for k=7

Schedule Finder for 20 Teams

```

1 import pulp, itertools, pandas as pd, numpy as np
2
3
4 names = [
5     "PalaLottomatica","Atlas Arena","Johan Cruyff Arena",
6     "Ta majdan Sports and Recreation Center","American Sports Center",
7     "Rogers Centre","Centro Ol mpico Juan Pablo Duarte",
8     "Coliseo de Puerto Rico Jos Miguel Agrelot",
9     "Arena da Baixada (Ligga Arena)","Estadio Luna Park",
10    "Estadio Metropolitano Roberto Mel ndez","Estadio Nacional del Per ",
11    "Accor Stadium (Stadium Australia)",
12    "Accor Stadium (Stadium Australia) 2","Sinan Erdem Dome",
13    "Beijing National Stadium ( Birds Nest)","Ariake Arena",
14    "Indoor Stadium Huamark",
15    "Kasarani Indoor Arena (Moi International Sports Centre)",
16    "Yaound Multipurpose Sports Complex",
17 ]
18 raw = [
19     [0,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,1,1,1],
20     [1,0,1,1,0,0,0,0,0,0,0,0,0,0,0,0,1,1,1,0,0,1],
21     [1,1,0,1,0,1,0,0,0,0,0,0,0,0,0,0,1,1,0,0,0,1],
22     [1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,0,0,1,1],
23     [0,0,0,0,0,1,1,1,0,0,1,1,0,1,0,0,1,0,0,1,0,0],
24     [0,0,1,0,1,0,1,1,0,0,1,1,0,0,0,1,1,0,0,0,1,0],
25     [0,0,0,0,1,1,0,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0],
26     [0,0,0,0,1,1,1,0,1,1,1,1,1,0,0,0,0,0,0,0,0,0],
27     [0,0,0,0,0,0,1,1,0,1,1,1,1,0,0,0,0,0,0,0,1],
28     [0,0,0,0,0,0,1,1,1,0,1,1,1,1,0,0,0,0,0,0,0],
29     [0,0,0,0,1,1,1,1,1,1,1,1,1,0,1,0,0,0,0,0,0],
30     [0,0,0,0,1,1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0],
31     [0,0,0,0,0,0,0,0,1,1,0,0,0,1,0,1,1,1,1,0,1],
32     [0,0,0,0,1,0,0,0,0,1,0,0,1,0,0,1,1,1,1,0,1],
33     [1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,1],
34     [0,1,1,1,0,0,0,0,0,0,0,0,0,1,1,0,0,1,1,0,0,0],
35     [0,1,0,0,1,1,0,0,0,0,0,0,0,1,1,0,1,0,1,0,0,0],
36     [1,0,0,0,0,0,0,0,0,0,0,0,0,1,1,1,1,1,0,1,0,1],
37     [1,0,0,1,0,0,0,0,0,0,0,0,0,1,1,1,0,0,1,0,1,1],
38     [1,1,1,1,0,0,0,0,1,0,0,0,0,0,1,0,0,0,1,0,1,0],
39 ]
40 adj = np.array(raw, dtype=int)
41 n = len(names)

```



```

42
43 assert adj.shape==(n,n)
44 assert (adj==adj.T).all()
45 assert np.all(np.diag(adj)==0)
46 assert (adj.sum(axis=1)==7).all(), "each node must have degree 7"
47
48
49 pairs = [(i,j) for i in range(n) for j in range(i+1,n) if adj[i,j]==1]
50
51
52 occurrences = [(i,j,k) for (i,j) in pairs for k in (0,1)]
53
54
55 D = list(range(240))
56 G = len(occurrences)
57 avg = G/len(D)
58 max_games_per_day = int(np.ceil(avg))
59
60
61 m = pulp.LpProblem("Alternating_HA_and_Gaps", pulp.LpMinimize)
62 m += 0
63
64
65 x = pulp.LpVariable.dicts("x", (occurrences, D), cat="Binary")
66
67
68 day_occ = {
69     occ: pulp.LpVariable(f"day_{occ[0]}_{occ[1]}_{occ[2]}", lowBound=0, upBound=239, cat="Continuous")
70     for occ in occurrences
71 }
72
73
74 h = {
75     occ: pulp.LpVariable(f"h_{occ[0]}_{occ[1]}_{occ[2]}", cat="Binary")
76     for occ in occurrences
77 }
78
79
80 for occ in occurrences:
81     m += pulp.lpSum(x[occ][d] for d in D) == 1
82
83
84 for occ in occurrences:
85     m += day_occ[occ] == pulp.lpSum(d * x[occ][d] for d in D)
86
87
88 for t in range(n):
89     for d in D:
90         m += pulp.lpSum(
91             x[(i,j,k)][d]
92             for (i,j,k) in occurrences
93             if i==t or j==t
94         ) <= 1
95
96
97 for (i,j) in pairs:
98     m += h[(i,j,0)] + h[(i,j,1)] == 1
99
100
101 for (i,j) in pairs:
102     m += day_occ[(i,j,0)] + 1 <= day_occ[(i,j,1)]
103
104
105 WINDOW = 5
106 for t in range(n):
107     occs_t = [o for o in occurrences if o[0]==t or o[1]==t]
108     for start in range(len(D)-WINDOW+1):
109         window_days = D[start:start+WINDOW]
110         m += pulp.lpSum(x[occ][d] for occ in occs_t for d in window_days) <= 1
111
112
113 for d in D:
114     m += pulp.lpSum(x[occ][d] for occ in occurrences) <= max_games_per_day

```

```
115
116
117 solver = pulp.PULP_CBC_CMD(msg=True, timeLimit=300)
118 m.solve(solver)
119 print("Status:", pulp.LpStatus[m.status])
120
121
122 schedule = []
123 for (i,j,k) in occurrences:
124     d = int(pulp.value(day_occ[(i,j,k)])) + 1
125     home = i if pulp.value(h[(i,j,k)]) > 0.5 else j
126     away = j if home==i else i
127     schedule.append((d, names[home], names[away], k+1))
128
129 df = pd.DataFrame(schedule, columns=["Day", "Home", "Away", "Meeting#"])
130 df = df.sort_values("Day")
131 df.to_csv("final_schedule.csv", index=False)
132 print("Wrote final_schedule.csv")
133
134
135
136
137 country_raw = [
138     "Italy", "Poland", "Netherlands", "Serbia", "USA",
139     "Canada", "Dominican Republic", "Puerto Rico",
140     "Brazil", "Argentina", "Colombia", "Peru",
141     "Australia 1", "Australia 2", "T rkiye", "China",
142     "Japan", "Thailand", "Kenya", "Cameroon"
143 ]
144 country_map = {i: country_raw[i] for i in range(n)}
145
146
147 schedule = []
148 for (i, j, k) in occurrences:
149     d = int(pulp.value(day_occ[(i, j, k)])) + 1
150     home = i if pulp.value(h[(i, j, k)]) > 0.5 else j
151     away = j if home == i else i
152     schedule.append((d, country_map[home], country_map[away], k + 1))
153
154
155 df = pd.DataFrame(schedule, columns=["Day", "HomeCountry", "AwayCountry", "Meeting#"])
156 df = df.sort_values("Day")
157 df.to_csv("final_schedule_by_country.csv", index=False)
158 print("Wrote final_schedule_by_country.csv")
```

2D Globe Visualization

```

1 import numpy as np
2 import pandas as pd
3 import plotly.graph_objs as go
4 from pyproj import Geod
5 import matplotlib.pyplot as plt
6 from collections import Counter
7 import re
8
9 venue_order = [
10     "PalaLottomatica", "Atlas Arena", "Johan Cruyff Arena", "Ta majdan Sports and Recreation Center",
11     "American Sports Center", "Rogers Centre", "Centro Ol mpico Juan Pablo Duarte",
12     "Coliseo de Puerto Rico Jos Miguel Agrelot", "Arena da Baixada (Ligga Arena)", "Estadio Luna Park",
13     "Estadio Metropolitano Roberto Mel ndez", "Estadio Nacional del Per ",
14     "Accor Stadium (Stadium Australia)", "Accor Stadium (Stadium Australia) 2", "Sinan Erdem Dome",
15     "Beijing National Stadium ( Birds Nest)", "Ariake Arena", "Indoor Stadium Huamark",
16     "Kasarani Indoor Arena (Moi International Sports Centre)", "Yaound Multipurpose Sports Complex"
17 ]
18
19 data_dict = {
20     "PalaLottomatica": {"country": "italy", "lat": 41.8253, "lon": 12.4666},
21     "Atlas Arena": {"country": "poland", "lat": 51.75694, "lon": 19.42500},
22     "Johan Cruyff Arena": {"country": "netherlands", "lat": 52.314224, "lon": 4.941841},
23     "Ta majdan Sports and Recreation Center": {"country": "serbia", "lat": 44.80934, "lon": 20.47067},
24     "American Sports Center": {"country": "usa", "lat": 33.8102, "lon": -117.9055},
25     "Rogers Centre": {"country": "canada", "lat": 43.641796, "lon": -79.390083},
26     "Centro Ol mpico Juan Pablo Duarte": {"country": "dominican republic", "lat": 18.4774, "lon":
27     -69.9170},
28     "Coliseo de Puerto Rico Jos Miguel Agrelot": {"country": "puerto rico", "lat": 18.4277, "lon":
29     -66.0614},
30     "Arena da Baixada (Ligga Arena)": {"country": "brazil", "lat": -25.4481, "lon": -49.2769},
31     "Estadio Luna Park": {"country": "argentina", "lat": -34.602032, "lon": -58.369022},
32     "Estadio Metropolitano Roberto Mel ndez": {"country": "colombia", "lat": 10.9269, "lon": -74.8006},
33     "Estadio Nacional del Per ": {"country": "peru", "lat": -12.067278, "lon": -77.033722},
34     "Accor Stadium (Stadium Australia)": {"country": "australia", "lat": -33.847057, "lon": 151.063156},
35     "Accor Stadium (Stadium Australia) 2": {"country": "australia", "lat": -33.837057, "lon": 151.073156},
36     "Sinan Erdem Dome": {"country": "t rkiye", "lat": 40.9884, "lon": 28.8539},
37     "Beijing National Stadium ( Birds Nest)": {"country": "china", "lat": 39.992935, "lon": 116.396446},
38     "Ariake Arena": {"country": "japan", "lat": 35.6406, "lon": 139.7971},
39     "Indoor Stadium Huamark": {"country": "thailand", "lat": 13.7563, "lon": 100.5018},
40     "Kasarani Indoor Arena (Moi International Sports Centre)": {"country": "kenya", "lat": -1.2195, "lon":
41     36.8867},
42     "Yaound Multipurpose Sports Complex": {"country": "cameroon", "lat": 3.8742, "lon": 11.5119}
43 }
44
45 data = []
46 for name in venue_order:
47     entry = data_dict[name].copy()
48     entry['name'] = name
49     data.append(entry)
50
51 df = pd.DataFrame(data)
52
53 raw_data = [
54     'PalaLottomatica: [0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1]',
55     'Atlas Arena: [1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1]',
56     'Johan Cruyff Arena: [1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1]',
57     'Ta majdan Sports and Recreation Center: [1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1]',
58     'American Sports Center: [0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0]',
59     'Rogers Centre: [0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0]',
60     'Centro Ol mpico Juan Pablo Duarte: [0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0]',
61     'Coliseo de Puerto Rico Jos Miguel Agrelot: [0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0]',
62     'Arena da Baixada (Ligga Arena): [0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1]',
63     'Estadio Luna Park: [0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]',
64     'Estadio Metropolitano Roberto Mel ndez: [0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0]',
65     'Estadio Nacional del Per : [0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]',
66     'Accor Stadium (Stadium Australia): [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0]',
67     'Accor Stadium (Stadium Australia) 2: [0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0]',
68     'Sinan Erdem Dome: [1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1]'
69 ]

```

```

66     'Beijing National Stadium (Birds Nest): [0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0,
67     0]',
68     'Ariake Arena: [0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0]',
69     'Indoor Stadium Huamark: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0]',
70     'Kasarani Indoor Arena (Moi International Sports Centre): [1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
71     1, 0, 0, 1, 0, 1]',
72     'Yaound Multipurpose Sports Complex: [1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0]'
73 ]
74 adjacency = []
75 for line in raw_data:
76     match = re.search(r'\s*([a-zA-Z0-9]+)\s*:', line)
77     if match:
78         row = list(map(int, match.group(1).split(',')))
79         adjacency.append(row)
80 adjacency = np.array(adjacency)
81 cmap = plt.get_cmap('tab20')
82 colors = [f'rgb({int(r*255)},{int(g*255)},{int(b*255)})' for r, g, b, _ in [cmap(i % 20) for i in range(
83     len(df))]]
84 geod = Geod(ellps='WGS84')
85 def interpolate_great_circle(lat1, lon1, lat2, lon2, n_points=50):
86     points = geod.npts(lon1, lat1, lon2, lat2, n_points)
87     points = [(lon1, lat1)] + points + [(lon2, lat2)]
88     lons, lats = zip(*points)
89     return lats, lons
90 venue_trace = go.Scattergeo(
91     lon=df['lon'],
92     lat=df['lat'],
93     # text=df['name'],
94     mode='markers+text',
95     marker=dict(size=6, color='red'),
96     textposition='top center'
97 )
98 )
99
100 uniform_color = 'black'
101
102 arc_traces = []
103 n = len(df)
104 for i in range(n):
105     for j in range(i+1, n):
106         if adjacency[i][j]:
107             lats, lons = interpolate_great_circle(df.loc[i, 'lat'], df.loc[i, 'lon'], df.loc[j, 'lat'], df
108             .loc[j, 'lon'])
109             arc_traces.append(go.Scattergeo(
110                 lon=lons,
111                 lat=lats,
112                 mode='lines',
113                 line=dict(width=1.5, color=uniform_color),
114                 opacity=0.8,
115                 showlegend=False
116             ))
117 fig = go.Figure(data=[venue_trace] + arc_traces)
118 fig.update_layout(
119     title='Global Venue Network (7-Regular Graph)',
120     geo=dict(
121         projection_type='natural earth',
122         showland=True,
123         landcolor='rgb(243, 243, 243)',
124         showcountries=True,
125         countrycolor='rgb(204, 204, 204)'
126     ),
127     margin=dict(l=0, r=0, t=40, b=0)
128 )
129 fig.show()
130 for idx, row in enumerate(adjacency):
131     print(f"{df.loc[idx, 'name']} has {np.sum(row)} connections")

```

3D Globe Visualization

```

1 import numpy as np
2 import pandas as pd
3 import plotly.graph_objs as go
4
5 data = [
6     {"Venue": "PalaLottomatica", "Latitude": 41.8253, "Longitude": 12.4666},
7     {"Venue": "Atlas Arena", "Latitude": 51.75694, "Longitude": 19.42500},
8     {"Venue": "Johan Cruyff Arena", "Latitude": 52.314224, "Longitude": 4.941841},
9     {"Venue": "Ta majdan Sports and Recreation Center", "Latitude": 44.80917, "Longitude": 20.47278},
10    {"Venue": "American Sports Center", "Latitude": 33.8175, "Longitude": -117.9136},
11    {"Venue": "Rogers Centre", "Latitude": 43.641796, "Longitude": -79.390083},
12    {"Venue": "Centro Ol mpico Juan Pablo Duarte", "Latitude": 18.4716, "Longitude": -69.9111},
13    {"Venue": "Coliseo de Puerto Rico Jos Miguel Agrelot", "Latitude": 18.4275, "Longitude": -66.0622},
14    {"Venue": "Arena da Baixada (Ligga Arena)", "Latitude": -25.4481, "Longitude": -49.2769},
15    {"Venue": "Estadio Luna Park", "Latitude": -34.602032, "Longitude": -58.369022},
16    {"Venue": "Estadio Metropolitano Roberto Mel ndez", "Latitude": 10.9269, "Longitude": -74.8006},
17    {"Venue": "Estadio Nacional del Per ", "Latitude": -12.067278, "Longitude": -77.033722},
18    {"Venue": "Accor Stadium (Stadium Australia)", "Latitude": -33.847057, "Longitude": 151.063156},
19    {"Venue": "Accor Stadium (Stadium Australia)", "Latitude": -33.847057, "Longitude": 151.063156},
20    {"Venue": "Sinan Erdem Dome", "Latitude": 40.9884, "Longitude": 28.8539},
21    {"Venue": "Beijing National Stadium (Birds Nest)", "Latitude": 39.992935, "Longitude": 116.396446},
22    {"Venue": "Ariake Arena", "Latitude": 35.6406, "Longitude": 139.7971},
23    {"Venue": "Indoor Stadium Huamark", "Latitude": 13.7563, "Longitude": 100.5018},
24    {"Venue": "Kasarani Indoor Arena (Moi International Sports Centre)", "Latitude": -1.2195, "Longitude": 36.8867},
25    {"Venue": "Yaound Multipurpose Sports Complex", "Latitude": 3.8742, "Longitude": 11.5119},
26 ]
27
28 df = pd.DataFrame(data)
29
30 lat = np.deg2rad(df['Latitude'].values)
31 lon = np.deg2rad(df['Longitude'].values)
32 r = 1
33
34 x = r * np.cos(lat) * np.cos(lon)
35 y = r * np.cos(lat) * np.sin(lon)
36 z = r * np.sin(lat)
37
38 theta, phi = np.mgrid[0:np.pi:100j, 0:2*np.pi:100j]
39 xs = r * np.sin(theta) * np.cos(phi)
40 ys = r * np.sin(theta) * np.sin(phi)
41 zs = r * np.cos(theta)
42
43 earth = go.Surface(x=xs, y=ys, z=zs, colorscale='Earth', opacity=0.7, showscale=False)
44
45 venues = go.Scatter3d(
46     x=x, y=y, z=z,
47     mode='markers+text',
48     marker=dict(size=5, color='red'),
49     # text=df['Venue'],
50     textposition='top center'
51 )
52
53 def great_circle_arc(x1, y1, z1, x2, y2, z2, n_points=50):
54     v1 = np.array([x1, y1, z1])
55     v2 = np.array([x2, y2, z2])
56     angles = np.linspace(0, 1, n_points)
57     arc = []
58     for t in angles:
59         v = v1 * np.sin((1-t)*np.arccos(np.dot(v1, v2))) + v2 * np.sin(t*np.arccos(np.dot(v1, v2)))
60         v = v / np.sin(np.arccos(np.dot(v1, v2)))
61         arc.append(v)
62     arc = np.array(arc)
63     return arc[:,0], arc[:,1], arc[:,2]
64
65 lines = []
66 for i in range(len(x)):
67     for j in range(i+1, len(x)):
68         xs_arc, ys_arc, zs_arc = great_circle_arc(x[i], y[i], z[i], x[j], y[j], z[j])
69         lines.append(go.Scatter3d(
70             x=xs_arc, y=ys_arc, z=zs_arc,
71             mode='lines',
72             line=dict(color='blue', width=1),

```

```

73         opacity=0.3,
74         showlegend=False
75     ))
76
77 fig = go.Figure(data=[earth, venues] + lines)
78 fig.update_layout(
79     scene=dict(
80         xaxis=dict(visible=False),
81         yaxis=dict(visible=False),
82         zaxis=dict(visible=False),
83         aspectmode='data'
84     ),
85     margin=dict(l=0, r=0, t=0, b=0)
86 )
87 fig.show()

```

Table 12: 20 Selected Teams for the Global Sports League

Day	Home Country	Away Country	Occurrence
1	Australia 1	Kenya	1
2	Netherlands	Türkiye	1
3	Colombia	Brazil	1
4	Thailand	Japan	1
5	Poland	Italy	1
6	USA	Australia 2	1
9	Dominican Republic	Argentina	1
10	Kenya	Serbia	1
11	USA	Japan	1
12	Türkiye	Italy	1
14	Australia 1	China	1
15	Australia 2	Kenya	1
17	USA	Dominican Republic	1
18	Japan	Poland	1
19	Serbia	China	1
20	Australia 2	Thailand	1
21	Argentina	Brazil	1
22	Netherlands	Cameroon	1
24	Puerto Rico	Peru	1
25	Serbia	Italy	1
26	China	Japan	1
27	Australia 2	USA	2
31	Netherlands	Serbia	1
32	Poland	Japan	2
35	Italy	Türkiye	2
36	Serbia	Kenya	2
37	Japan	China	2
38	Puerto Rico	Dominican Republic	1
40	Cameroon	Türkiye	1
43	China	Australia 1	2
44	Italy	Netherlands	1

45	Dominican Republic	Colombia	1
47	Canada	USA	1
48	Serbia	Poland	1
49	Australia 1	Brazil	1
50	Thailand	Türkiye	1
54	Peru	Puerto Rico	2
55	Serbia	Türkiye	1
57	Brazil	Australia 1	2
58	Japan	Australia 2	1
59	Thailand	China	1
60	Serbia	Netherlands	2
62	Brazil	Peru	1
63	Colombia	Dominican Republic	2
67	Peru	Canada	1
70	Argentina	Puerto Rico	1
71	Kenya	Türkiye	1
72	Japan	Australia 1	1
73	Peru	Brazil	2
76	Cameroon	Kenya	1
78	Brazil	Dominican Republic	1
79	China	Thailand	2
82	Australia 1	Australia 2	1
85	USA	Peru	1
86	Canada	Dominican Republic	1
88	Australia 1	Argentina	1
89	Japan	Thailand	2
91	Dominican Republic	Canada	2
93	Cameroon	Italy	1
95	Thailand	Kenya	1
96	Australia 1	Japan	2
97	Puerto Rico	Canada	1
98	Argentina	Australia 2	1
102	Kenya	Thailand	2
103	Dominican Republic	Brazil	2
104	Colombia	Puerto Rico	1
105	Netherlands	Poland	1
107	Peru	Argentina	1
108	Thailand	Australia 2	2
109	Dominican Republic	Puerto Rico	2
111	USA	Colombia	1
113	Argentina	Australia 1	2
114	Türkiye	Kenya	2
115	Italy	Thailand	1
116	Australia 2	Japan	2

119	Netherlands	Canada	1
120	Thailand	Italy	2
121	Japan	USA	2
124	Brazil	Colombia	2
125	Canada	Netherlands	2
126	Kenya	Australia 1	2
131	Brazil	Cameroon	1
132	Italy	Kenya	1
133	Türkiye	Thailand	2
134	China	Netherlands	1
139	Argentina	Dominican Republic	2
140	Poland	Cameroon	1
142	USA	Puerto Rico	1
143	Italy	Serbia	2
146	Cameroon	Netherlands	2
147	Japan	Canada	1
148	Dominican Republic	Peru	1
150	Australia 1	Thailand	1
152	Puerto Rico	Colombia	2
153	Cameroon	Serbia	1
154	Dominican Republic	USA	2
157	Puerto Rico	Brazil	1
159	China	Poland	1
161	Netherlands	Italy	2
164	Türkiye	Poland	1
167	Peru	Dominican Republic	2
170	Australia 2	Argentina	2
174	Puerto Rico	USA	2
178	Türkiye	Netherlands	2
179	Canada	Japan	2
180	Kenya	Cameroon	2
184	Colombia	Argentina	1
187	Poland	Serbia	2
191	Canada	Peru	2
193	Serbia	Cameroon	2
196	Argentina	Colombia	2
197	Kenya	Italy	2
198	Türkiye	Serbia	2
199	Poland	China	2
202	Peru	USA	2
204	Kenya	Australia 2	2
206	Colombia	Canada	1
207	Poland	Türkiye	2
212	Colombia	Peru	1

214	China	Australia 2	1
215	Brazil	Argentina	2
216	Italy	Cameroon	2
217	Poland	Netherlands	2
218	Canada	Colombia	2
219	Australia 2	China	2
220	Argentina	Peru	2
221	Thailand	Australia 1	2
222	Türkiye	Cameroon	2
226	Brazil	Puerto Rico	2
227	Italy	Poland	2
228	USA	Canada	2
230	China	Serbia	2
232	Cameroon	Brazil	2
233	Colombia	USA	2
234	Puerto Rico	Argentina	2
235	Netherlands	China	2
237	Australia 2	Australia 1	2
238	Peru	Colombia	2
239	Cameroon	Poland	2
240	Canada	Puerto Rico	2

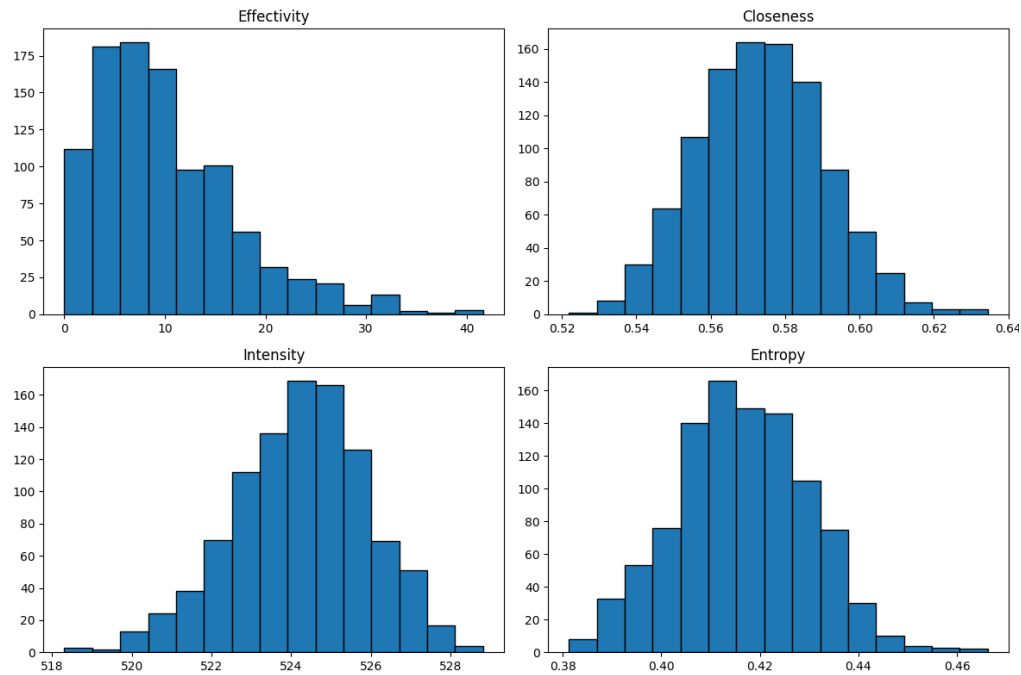


Figure 15: Round Robin Results

Schedule Simulator (Effectivity and Attractiveness)

```

1 import random
2 import numpy as np
3 import math
4 from scipy.stats import norm
5 from collections import defaultdict
6
7 K = 45
8 C1, C2, C3, C4, C5 = -1.06, -0.394, 0, 0.394, 1.06
9 R_VALUES = [2.0, 1.5, 1.0, -1.0, -1.5, -2.0]
10
11 def get_probabilities(delta):
12     P1 = norm.cdf(C1 + delta)
13     P2 = norm.cdf(C2 + delta) - P1
14     P3 = norm.cdf(C3 + delta) - norm.cdf(C2 + delta)
15     P4 = norm.cdf(C4 + delta) - norm.cdf(C3 + delta)
16     P5 = norm.cdf(C5 + delta) - norm.cdf(C4 + delta)
17     P6 = 1 - norm.cdf(C5 + delta)
18     return [P1, P2, P3, P4, P5, P6]
19
20 def expected_result(delta):
21     return sum(R * P for R, P in zip(R_VALUES, get_probabilities(delta)))
22
23 def simulate_match(team_a, team_b, strengths):
24     delta = 8 * (strengths[team_a] - strengths[team_b]) / 1000
25     p_win = sum(get_probabilities(delta)[:3])
26     return team_a if random.random() < p_win else team_b
27
28 def update_ratings(team_a, team_b, outcome, strengths):
29     delta = 8 * (strengths[team_a] - strengths[team_b]) / 1000
30     E = expected_result(delta)
31     change = K * (outcome - E) / 8
32     strengths[team_a] += change
33     strengths[team_b] -= change
34
35 def safe_weighted_inversion_score(reference, ranking):
36     score, pos = 0, {t: i for i, t in enumerate(ranking)}
37     for i, ti in enumerate(reference):
38         if ti not in pos: continue
39         for tj in reference[i+1:]:

```

```

40         if tj not in pos: continue
41         if pos[ti] > pos[tj]:
42             score += sum(1 / math.log(k + 1) for k in range(pos[tj] + 1, pos[ti] + 1))
43     return score
44
45 team_strengths = {
46     "Italy": 437.03, "Poland": 349.75, "Netherlands": 283.99, "Serbia": 280.42,
47     "USA": 362.27, "Canada": 284.76, "Dominican Republic": 260.54, "Puerto Rico": 182.58,
48     "Brazil": 407.09, "Argentina": 180.96, "Colombia": 152.37, "Peru": 93.05,
49     "Australia": 44.13, "Australia B": 40.13, "Turkiye": 352.61, "China": 350.3,
50     "Japan": 325.18, "Thailand": 194.91, "Kenya": 152.15, "Cameroon": 85.69,
51 }
52
53 manual_schedule = [
54     (1, "Australia", "Kenya"),
55     (2, "Netherlands", "Turkiye"),
56     (3, "Colombia", "Brazil"),
57     (4, "Thailand", "Japan"),
58     (5, "Poland", "Italy"),
59     (6, "USA", "Australia B"),
60     (9, "Dominican Republic", "Argentina"),
61     (10, "Kenya", "Serbia"),
62     (11, "USA", "Japan"),
63     (12, "Turkiye", "Italy"),
64     (14, "Australia", "China"),
65     (15, "Australia B", "Kenya"),
66     (17, "USA", "Dominican Republic"),
67     (18, "Japan", "Poland"),
68     (19, "Serbia", "China"),
69     (20, "Australia B", "Thailand"),
70     (21, "Argentina", "Brazil"),
71     (22, "Netherlands", "Cameroon"),
72     (24, "Puerto Rico", "Peru"),
73     (25, "Serbia", "Italy"),
74     (26, "China", "Japan"),
75     (27, "Australia B", "USA"),
76     (31, "Netherlands", "Serbia"),
77     (32, "Poland", "Japan"),
78     (35, "Italy", "Turkiye"),
79     (36, "Serbia", "Kenya"),
80     (37, "Japan", "China"),
81     (38, "Puerto Rico", "Dominican Republic"),
82     (40, "Cameroon", "Turkiye"),
83     (43, "China", "Australia"),
84     (44, "Italy", "Netherlands"),
85     (45, "Dominican Republic", "Colombia"),
86     (47, "Canada", "USA"),
87     (48, "Serbia", "Poland"),
88     (49, "Australia", "Brazil"),
89     (50, "Thailand", "Turkiye"),
90     (54, "Peru", "Puerto Rico"),
91     (55, "Serbia", "Turkiye"),
92     (57, "Brazil", "Australia"),
93     (58, "Japan", "Australia B"),
94     (59, "Thailand", "China"),
95     (60, "Serbia", "Netherlands"),
96     (62, "Brazil", "Peru"),
97     (63, "Colombia", "Dominican Republic"),
98     (67, "Peru", "Canada"),
99     (70, "Argentina", "Puerto Rico"),
100    (71, "Kenya", "Turkiye"),
101    (72, "Japan", "Australia"),
102    (73, "Peru", "Brazil"),
103    (76, "Cameroon", "Kenya"),
104    (78, "Brazil", "Dominican Republic"),
105    (79, "China", "Thailand"),
106    (82, "Australia", "Australia B"),
107    (85, "USA", "Peru"),
108    (86, "Canada", "Dominican Republic"),
109    (88, "Australia", "Argentina"),
110    (89, "Japan", "Thailand"),
111    (91, "Dominican Republic", "Canada"),
112    (93, "Cameroon", "Italy"),

```

```

113 (95, "Thailand", "Kenya"),
114 (96, "Australia", "Japan"),
115 (97, "Puerto Rico", "Canada"),
116 (98, "Argentina", "Australia B"),
117 (102, "Kenya", "Thailand"),
118 (103, "Dominican Republic", "Brazil"),
119 (104, "Colombia", "Puerto Rico"),
120 (105, "Netherlands", "Poland"),
121 (107, "Peru", "Argentina"),
122 (108, "Thailand", "Australia B"),
123 (109, "Dominican Republic", "Puerto Rico"),
124 (111, "USA", "Colombia"),
125 (113, "Argentina", "Australia"),
126 (114, "Turkiye", "Kenya"),
127 (115, "Italy", "Thailand"),
128 (116, "Australia B", "Japan"),
129 (119, "Netherlands", "Canada"),
130 (120, "Thailand", "Italy"),
131 (121, "Japan", "USA"),
132 (124, "Brazil", "Colombia"),
133 (125, "Canada", "Netherlands"),
134 (126, "Kenya", "Australia"),
135 (131, "Brazil", "Cameroon"),
136 (132, "Italy", "Kenya"),
137 (133, "Turkiye", "Thailand"),
138 (134, "China", "Netherlands"),
139 (139, "Argentina", "Dominican Republic"),
140 (140, "Poland", "Cameroon"),
141 (142, "USA", "Puerto Rico"),
142 (143, "Italy", "Serbia"),
143 (146, "Cameroon", "Netherlands"),
144 (147, "Japan", "Canada"),
145 (148, "Dominican Republic", "Peru"),
146 (150, "Australia", "Thailand"),
147 (152, "Puerto Rico", "Colombia"),
148 (153, "Cameroon", "Serbia"),
149 (154, "Dominican Republic", "USA"),
150 (157, "Puerto Rico", "Brazil"),
151 (159, "China", "Poland"),
152 (161, "Netherlands", "Italy"),
153 (164, "Turkiye", "Poland"),
154 (167, "Peru", "Dominican Republic"),
155 (170, "Australia B", "Argentina"),
156 (174, "Puerto Rico", "USA"),
157 (178, "Turkiye", "Netherlands"),
158 (179, "Canada", "Japan"),
159 (180, "Kenya", "Cameroon"),
160 (184, "Colombia", "Argentina"),
161 (187, "Poland", "Serbia"),
162 (191, "Canada", "Peru"),
163 (193, "Serbia", "Cameroon"),
164 (196, "Argentina", "Colombia"),
165 (197, "Kenya", "Italy"),
166 (198, "Turkiye", "Serbia"),
167 (199, "Poland", "China"),
168 (202, "Peru", "USA"),
169 (204, "Kenya", "Australia B"),
170 (206, "Colombia", "Canada"),
171 (207, "Poland", "Turkiye"),
172 (212, "Colombia", "Peru"),
173 (214, "China", "Australia B"),
174 (215, "Brazil", "Argentina"),
175 (216, "Italy", "Cameroon"),
176 (217, "Poland", "Netherlands"),
177 (218, "Canada", "Colombia"),
178 (219, "Australia B", "China"),
179 (220, "Argentina", "Peru"),
180 (221, "Thailand", "Australia"),
181 (222, "Turkiye", "Cameroon"),
182 (226, "Brazil", "Puerto Rico"),
183 (227, "Italy", "Poland"),
184 (228, "USA", "Canada"),
185 (230, "China", "Serbia"),

```

```

186     (232, "Cameroon", "Brazil"),
187     (233, "Colombia", "USA"),
188     (234, "Puerto Rico", "Argentina"),
189     (235, "Netherlands", "China"),
190     (237, "Australia B", "Australia"),
191     (238, "Peru", "Colombia"),
192     (239, "Cameroon", "Poland"),
193     (240, "Canada", "Puerto Rico"),
194 ]
195
196 fixed_manual_schedule = [(a, b) for (_, a, b) in manual_schedule]
197
198 def swiss_pairing(teams, results):
199     wins = defaultdict(int)
200     for w, l in results:
201         wins[w] += 1
202     sorted_teams = sorted(teams, key=lambda t: (-wins[t], -team_strengths[t]))
203     used, pairings = set(), []
204     for i, t in enumerate(sorted_teams):
205         if t in used: continue
206         for opp in sorted_teams[i+1:]:
207             if opp not in used and (t, opp) not in results and (opp, t) not in results:
208                 pairings.append((t, opp))
209                 used |= {t, opp}
210                 break
211     return pairings
212
213 def simulate_swiss_top8(top8, strengths, rounds=5):
214     local = strengths.copy()
215     results, sched = [], []
216     for _ in range(rounds):
217         matches = swiss_pairing(top8, results)
218         for a, b in matches:
219             winner = simulate_match(a, b, local)
220             results.append((winner, b if winner == a else a))
221             sched.append((a, b))
222             update_ratings(a, b, 2 if winner == a else -2, local)
223     final = sorted(top8, key=lambda t: (-sum(1 for w, _ in results if w == t), -local[t]))
224     return final, sched
225
226 def simulate_round_robin_top8(top8, strengths):
227     local = strengths.copy()
228     results = []
229     schedule = []
230
231     points = defaultdict(int)
232     for i in range(len(top8)):
233         for j in range(i + 1, len(top8)):
234             A, B = top8[i], top8[j]
235             winner = simulate_match(A, B, local)
236             loser = B if winner == A else A
237             update_ratings(A, B, 2 if winner == A else -2, local)
238             points[winner] += 2
239             results.append((winner, loser))
240             schedule.append((A, B))
241
242     final_ranking = sorted(top8, key=lambda t: (-points[t], -local[t]))
243     return final_ranking[:8], schedule
244
245 def simulate_double_elimination_top8(top8, strengths):
246     from collections import deque
247
248     sched = []
249     results = []
250     winners, losers = deque(top8), deque()
251     local = strengths.copy()
252
253     def match(A, B):
254         if A is None: return B
255         if B is None: return A
256         winner = simulate_match(A, B, local)
257         results.append((winner, B if winner == A else A))
258         sched.append((A, B))

```

```

259     update_ratings(A, B, 2 if winner == A else -2, local)
260     return winner
261
262 while len(winners) + len(losers) > 2:
263     next_winners = deque()
264     while winners:
265         A = winners.popleft()
266         B = winners.popleft() if winners else None
267         win = match(A, B)
268         lose = B if win == A else A
269         next_winners.append(win)
270         if lose: losers.append(lose)
271     winners = next_winners
272
273     next_losers = deque()
274     while len(losers) > 1:
275         A = losers.popleft()
276         B = losers.popleft()
277         win = match(A, B)
278         next_losers.append(win)
279     if losers:
280         next_losers.append(losers.popleft())
281     losers = next_losers
282
283 final1 = winners.pop() if winners else None
284 final2 = losers.pop() if losers else None
285 champion = match(final1, final2)
286
287 all_played = list({t for match in sched for t in match})
288 final_rank = sorted(all_played, key=lambda t: -local[t])
289 return final_rank[:8], sched
290
291 def get_final_top8_from_combined_run(strengths, schedule):
292     local = strengths.copy()
293     for a, b in schedule:
294         winner = simulate_match(a, b, local)
295         update_ratings(a, b, 2 if winner == a else -2, local)
296     top8 = sorted(local, key=local.get, reverse=True)[:8]
297     final_rank, _ = simulate_swiss_top8(top8, local)
298     return final_rank[:8]
299
300
301
302 def get_placement_and_schedule(strengths):
303     local = strengths.copy()
304     for a, b in fixed_manual_schedule:
305         winner = simulate_match(a, b, local)
306         update_ratings(a, b, 2 if winner == a else -2, local)
307     top8 = sorted(local, key=local.get, reverse=True)[:8]
308     final_rank, swiss_schedule = simulate_swiss_top8(top8, local)
309     return final_rank[:8], fixed_manual_schedule + swiss_schedule, local
310
311 def get_placement_and_schedule_knockout(strengths):
312     local = strengths.copy()
313     for a, b in fixed_manual_schedule:
314         winner = simulate_match(a, b, local)
315         update_ratings(a, b, 2 if winner == a else -2, local)
316     top8 = sorted(local, key=local.get, reverse=True)[:8]
317     final_rank, knockout_schedule = simulate_double_elimination_top8(top8, local)
318     return final_rank, fixed_manual_schedule + knockout_schedule, local
319
320 def get_placement_and_schedule_roundrobin(strengths):
321     local = strengths.copy()
322     for a, b in fixed_manual_schedule:
323         winner = simulate_match(a, b, local)
324         update_ratings(a, b, 2 if winner == a else -2, local)
325     top8 = sorted(local, key=local.get, reverse=True)[:8]
326     final_rank, rr_schedule = simulate_round_robin_top8(top8, local)
327     return final_rank, fixed_manual_schedule + rr_schedule, local
328
329
330 def compute_alignment_with_updated_strengths(get_placement_func, base_strengths, sims=10):
331     return np.mean([

```

```

332     safe_weighted_inversion_score(
333         sorted(strengths := get_placement_func(base_strengths.copy())[2],
334             key=strengths.get, reverse=True)[:8],
335         get_placement_func(base_strengths.copy())[0]
336     )
337     for _ in range(sims)
338 ]
339
340 def compute_attractiveness(schedule, strengths):
341     closeness_list, intensity_list, entropy_list = [], [], []
342     for team_A, team_B in schedule:
343         r1, r2 = strengths[team_A], strengths[team_B]
344         closeness = 1 - abs(r1 - r2) / max(r1, r2)
345         intensity = r1 + r2
346         delta = 8 * (r1 - r2) / 1000
347         p = sum(get_probabilities(delta)[:3])
348         entropy = 0 if p in (0, 1) else -p * math.log(p) - (1 - p) * math.log(1 - p)
349         closeness_list.append(closeness)
350         intensity_list.append(intensity)
351         entropy_list.append(entropy)
352     return np.mean(closeness_list), np.mean(intensity_list), np.mean(entropy_list)
353
354 import matplotlib.pyplot as plt
355
356 def simulate_all_metrics(get_placement_func, base_strengths, sims=100):
357     alignments, closenesses, intensities, entropies = [], [], [], []
358     for _ in range(sims):
359         ranking, schedule, final_strengths = get_placement_func(base_strengths.copy())
360         final_sorted = sorted(final_strengths, key=final_strengths.get, reverse=True)
361         alignment = safe_weighted_inversion_score(final_sorted[:8], ranking[:8])
362         alignments.append(alignment)
363
364         close, intense, ent = compute_attractiveness(schedule, final_strengths)
365         closenesses.append(close)
366         intensities.append(intense)
367         entropies.append(ent)
368     return alignments, closenesses, intensities, entropies
369
370 alignments, closenesses, intensities, entropies = simulate_all_metrics(get_placement_and_schedule,
371     team_strengths, sims=1000)
372
373 # fig, axs = plt.subplots(2, 2, figsize=(12, 8))
374 # axs[0, 0].hist(alignments, bins=15, edgecolor='black')
375 # axs[0, 0].set_title("Effectivity")
376
377 # axs[0, 1].hist(closenesses, bins=15, edgecolor='black')
378 # axs[0, 1].set_title("Closeness")
379
380 # axs[1, 0].hist(intensities, bins=15, edgecolor='black')
381 # axs[1, 0].set_title("Intensity")
382
383 # axs[1, 1].hist(entropies, bins=15, edgecolor='black')
384 # axs[1, 1].set_title("Entropy")
385
386 # plt.tight_layout()
387 # plt.show()

```

9.3 Problem 3

Table 13: 24 Selected Teams for the Global Sports League with Coordinates

Rank	Team	Continent	Venue	Coordinates
1	Italy	Europe	PalaLottomatica	41.8253° N, 12.4666° E
2	Brazil	South America	Arena da Baixada	25.4481° S, 49.2769° W

3	USA	North America	American Sports Center	33.8102° N, 117.9055° W
4	Türkiye	Asia	Sinan Erdem Dome	40.9884° N, 28.8539° E
5	China	Asia	Beijing National Stadium	39.992935° N, 116.396446° E
6	Poland	Europe	Atlas Arena	51.75694° N, 19.42500° E
7	Japan	Asia	Ariake Arena	35.6406° N, 139.7971° E
8	Canada	North America	Rogers Centre	43.641796° N, 79.390083° W
9	Netherlands	Europe	Johan Cruyff Arena	52.314224° N, 4.941841° E
10	Serbia	Europe	Tašmajdan Sports and Recreation Center	44.80934° N, 20.47067° E
11	Dominican Republic	North America	Centro Olímpico Juan Pablo Duarte	18.4774° N, 69.917° W
12	Germany	Europe	Max-Schmeling-Halle	52.54495° N, 13.40440° E
13	Thailand	Asia	Indoor Stadium Huamark	13.7563° N, 100.5018° E
14	Puerto Rico	North America	Coliseo de Puerto Rico José Miguel Agrelot	18.4277° N, 66.0614° W
15	Argentina	South America	Estadio Luna Park	34.602032° S, 58.369022° W
16	Colombia	South America	Estadio Metropolitano Roberto Meléndez	10.9269° N, 74.8006° W
17	Kenya	Africa	Kasarani Indoor Arena	1.2195° S, 36.8867° E
18	Cuba	North America	Ciudad Deportiva Coliseum	23.10245° N, 82.39370° W
19	Peru	South America	Estadio Nacional del Perú	12.067278° S, 77.033722° W
20	Cameroon	Africa	Yaoundé Multipurpose Sports Complex	3.8742° N, 11.5119° E
21	Egypt	Africa	Cairo Stadium Indoor Halls Complex	30.06989° N, 31.30722° E
22	Australia	Oceania	Accor Stadium	33.847057° S, 151.063156° E
23	Australia	Oceania	Accor Stadium	33.847057° S, 151.063156° E
24	Algeria	Africa	Salle Harcha Hassen	36.75591° N, 3.05472° E

STD and Distance Minimizer for 24 Teams

```

1 import pandas as pd
2 import numpy as np
3 from geopy.distance import geodesic
4
5

```



```

6 data = [
7     {"name": "PalaLottomatica", "lat": 41.8253, "lon": 12.4666},
8     {"name": "Atlas Arena", "lat": 51.75694, "lon": 19.42500},
9     {"name": "Johan Cruyff Arena", "lat": 52.314224, "lon": 4.941841},
10    {"name": "Ta majdan Sports Center", "lat": 44.80934, "lon": 20.47067},
11    {"name": "American Sports Center", "lat": 33.8102, "lon": -117.9055},
12    {"name": "Rogers Centre", "lat": 43.641796, "lon": -79.390083},
13    {"name": "Centro Ol. Juan P. Duarte", "lat": 18.4774, "lon": -69.9170},
14    {"name": "Coliseo PR Agrelot", "lat": 18.4277, "lon": -66.0614},
15    {"name": "Arena da Baixada", "lat": -25.4481, "lon": -49.2769},
16    {"name": "Estadio Luna Park", "lat": -34.602032, "lon": -58.369022},
17    {"name": "Estadio Metropolitano", "lat": 10.9269, "lon": -74.8006},
18    {"name": "Estadio Nac Peru", "lat": -12.067278, "lon": -77.033722},
19    {"name": "Accor Stadium AU1", "lat": -33.847057, "lon": 151.063156},
20    {"name": "Accor Stadium AU2", "lat": -33.847057, "lon": 151.063156},
21    {"name": "Sinan Erdem Dome", "lat": 40.9884, "lon": 28.8539},
22    {"name": "Beijing Natl Stad", "lat": 39.992935, "lon": 116.396446},
23    {"name": "Ariake Arena", "lat": 35.6406, "lon": 139.7971},
24    {"name": "Indoor Stad Huamark", "lat": 13.7563, "lon": 100.5018},
25    {"name": "Kasarani Indoor", "lat": -1.2195, "lon": 36.8867},
26    {"name": "Yaounde Complex", "lat": 3.8742, "lon": 11.5119},
27    {"name": "Max-Schmeling-Halle", "lat": 52.54495, "lon": 13.40440},
28    {"name": "Ciudad Deportiva Coliseum", "lat": 23.10245, "lon": -82.39370},
29    {"name": "Cairo Stadium Indoor", "lat": 30.06989, "lon": 31.30722},
30    {"name": "Salle Harcha Hassen", "lat": 36.75591, "lon": 3.05472}
31 ]
32
33 df = pd.DataFrame(data)
34 n = len(df)
35
36
37 dist = np.zeros((n, n))
38 for i in range(n):
39     for j in range(n):
40         dist[i, j] = geodesic((df.loc[i, "lat"], df.loc[i, "lon"]),
41                               (df.loc[j, "lat"], df.loc[j, "lon"])).km
42
43 std_devs = []
44 tot_dists = []
45
46 for k in range(1, n):
47     adj = np.zeros((n, n), int)
48     total_distance = 0.0
49
50
51     for i in range(n):
52         nn = np.argsort(dist[i])[1:k+1]
53         for j in nn:
54             adj[i, j] = 1
55             total_distance += dist[i, j]
56
57
58     P = adj.astype(float)
59     P = P / P.sum(axis=1, keepdims=True)
60
61
62     eigvals, eigvecs = np.linalg.eig(P.T)
63     idx1 = np.argmin(np.abs(eigvals - 1))
64     steady = np.real(eigvecs[:, idx1])
65     steady = steady / steady.sum()
66
67     std_devs.append(np.std(steady))
68     tot_dists.append(total_distance)
69
70
71 print(" k | std() | directed-distance-sum (km)")
72 print("-----")
73 for k in range(1, n):
74     print(f"{k:2d} | {std_devs[k-1]:8.5f} | {tot_dists[k-1]:10.0f}")

```

Transition Matrix Finder When k=11 for 24 Teams

```

1 import pandas as pd
2 import numpy as np
3 from geopy.distance import geodesic
4 import pulp
5
6 data = [
7     {"country": "italy", "name": "PalaLottomatica", "lat": 41.8253, "lon": 12.4666},
8     {"country": "poland", "name": "Atlas Arena", "lat": 51.75694, "lon": 19.42500},
9     {"country": "netherlands", "name": "Johan Cruyff Arena", "lat": 52.314224, "lon": 4.941841},
10    {"country": "serbia", "name": "Ta majdan Sports and Recreation Center", "lat": 44.80934, "lon":
11    20.47067},
12    {"country": "usa", "name": "American Sports Center", "lat": 33.8102, "lon": -117.9055},
13    {"country": "canada", "name": "Rogers Centre", "lat": 43.641796, "lon": -79.390083},
14    {"country": "dominican republic", "name": "Centro Ol mpico Juan Pablo Duarte", "lat": 18.4774, "lon":
15    -69.9170},
16    {"country": "puerto rico", "name": "Coliseo de Puerto Rico Jos Miguel Agrelot", "lat": 18.4277, "lon":
17    -66.0614},
18    {"country": "brazil", "name": "Arena da Baixada (Ligga Arena)", "lat": -25.4481, "lon": -49.2769},
19    {"country": "argentina", "name": "Estadio Luna Park", "lat": -34.602032, "lon": -58.369022},
20    {"country": "colombia", "name": "Estadio Metropolitano Roberto Mel ndez", "lat": 10.9269, "lon":
21    -74.8006},
22    {"country": "peru", "name": "Estadio Nacional del Per ", "lat": -12.067278, "lon": -77.033722},
23    {"country": "australia", "name": "Accor Stadium (Stadium Australia)", "lat": -33.847057, "lon":
24    151.063156},
25    {"country": "australia2", "name": "Accor Stadium (Stadium Australia)", "lat": -33.847057, "lon":
26    151.063156},
27    {"country": "t rkiye", "name": "Sinan Erdem Dome", "lat": 40.9884, "lon": 28.8539},
28    {"country": "china", "name": "Beijing National Stadium (Birds Nest)", "lat": 39.992935, "lon":
29    116.396446},
30    {"country": "japan", "name": "Ariake Arena", "lat": 35.6406, "lon": 139.7971},
31    {"country": "thailand", "name": "Indoor Stadium Huamark", "lat": 13.7563, "lon": 100.5018},
32    {"country": "kenya", "name": "Kasarani Indoor Arena (Moi International Sports Centre)", "lat":
33    -1.2195, "lon": 36.8867},
34    {"country": "cameroon", "name": "Yaound Multipurpose Sports Complex", "lat": 3.8742, "lon":
35    11.5119},
36    {"name": "Max-Schmeling-Halle", "lat": 52.54495, "lon": 13.40440},
37    {"name": "Ciudad Deportiva Coliseum", "lat": 23.10245, "lon": -82.39370},
38    {"name": "Cairo Stadium Indoor", "lat": 30.06989, "lon": 31.30722},
39    {"name": "Salle Harcha Hassen", "lat": 36.75591, "lon": 3.05472}
40 ]
41 df = pd.DataFrame(data)
42 n = len(df)
43 k = 11
44
45 dist = {}
46 for i in range(n):
47     dist[i] = {}
48     coord_i = (df.loc[i, "lat"], df.loc[i, "lon"])
49     for j in range(n):
50         coord_j = (df.loc[j, "lat"], df.loc[j, "lon"])
51         dist[i][j] = round(geodesic(coord_i, coord_j).km, 2)
52
53 model = pulp.LpProblem("k_regular_graph", pulp.LpMinimize)
54
55 x = {}
56 for i in range(n):
57     for j in range(i+1, n):
58         x[(i, j)] = pulp.LpVariable(f"x_{i}_{j}", cat="Binary")
59
60 model += pulp.lpSum(dist[i][j] * x[(i, j)] for i in range(n) for j in range(i+1, n))
61
62 for i in range(n):
63     model += (
64         pulp.lpSum(
65             x[(min(i, j), max(i, j))] for j in range(n) if j != i
66         ) == k
67     )
68
69 model.solve(pulp.PULP_CBC_CMD(msg=False))
70
71 adj = np.zeros((n, n), dtype=int)
72 for (i, j), var in x.items():

```

```

65     if pulp.value(var) > 0.5:
66         adj[i, j] = adj[j, i] = 1
67
68 adj_df = pd.DataFrame(adj, index=df['name'], columns=df['name'])
69 print(adj_df)
70 adj_rows = [f"{row}: {adj_df.loc[row].values.tolist()}" for row in adj_df.index]
71 print(adj_rows)

```

Schedule Generator for 24 Teams

```

1  import pulp
2  import itertools
3  import pandas as pd
4  import numpy as np
5
6
7  names = [
8      "PalaLottomatica", "Atlas Arena", "Johan Cruyff Arena",
9      "Ta majdan Sports and Recreation Center", "American Sports Center",
10     "Rogers Centre", "Centro Ol mpico Juan Pablo Duarte",
11     "Coliseo de Puerto Rico Jos Miguel Agrelot",
12     "Arena da Baixada (Ligga Arena)", "Estadio Luna Park",
13     "Estadio Metropolitano Roberto Mel ndez", "Estadio Nacional del Per ",
14     "Accor Stadium (Stadium Australia)", "Accor Stadium (Stadium Australia) 2",
15     "Sinan Erdem Dome", "Beijing National Stadium ( Birds Nest)",
16     "Ariake Arena", "Indoor Stadium Huamark",
17     "Kasarani Indoor Arena (Moi International Sports Centre)",
18     "Yaound Multipurpose Sports Complex",
19     " M a x SchmelingHalle ",
20     "Ciudad Deportiva Coliseum",
21     "Cairo Stadium Indoor Halls",
22     "Salle Harcha Hassen"
23 ]
24 assert len(names) == 24
25
26
27 raw = [
28     [0,1,1,1,0,0,1,0,0,0,0,0,0,0,0,1,0,0,1,1,1,1,0,1,1],
29     [1,0,1,1,1,0,0,0,0,0,0,0,0,0,0,1,1,1,1,0,1,1,0,1,1],
30     [1,1,0,1,0,1,1,1,0,0,0,0,0,0,0,1,1,0,0,0,0,1,0,1,1],
31     [1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,1,1,0,1,1,1,1,0,1,1],
32     [0,0,0,0,0,1,1,1,1,1,1,1,1,1,1,0,0,1,0,0,0,0,1,0,0],
33     [0,0,1,0,1,0,1,1,1,1,1,1,1,0,0,0,0,1,0,0,0,1,1,0,0],
34     [1,0,1,0,1,1,0,1,1,1,1,1,0,0,0,0,0,0,0,0,0,1,0,1],
35     [0,0,1,0,1,1,1,0,1,1,1,1,0,0,0,0,0,0,0,1,0,1,0,1],
36     [0,0,0,0,1,1,1,1,0,1,1,1,1,1,0,0,0,0,0,1,0,1,0,0],
37     [0,0,0,0,1,1,1,1,1,0,1,1,1,1,0,0,0,0,0,1,0,1,0,0],
38     [0,0,0,0,1,1,1,1,1,1,0,1,1,1,0,0,0,1,0,0,0,0,1,0,0],
39     [0,0,0,0,1,1,1,1,1,1,1,0,1,1,0,0,0,0,0,1,0,1,0,0],
40     [0,0,0,0,1,0,0,0,1,1,1,1,0,1,0,1,1,1,1,0,0,1,0,0],
41     [0,0,0,0,1,0,0,0,1,1,1,1,1,0,0,1,1,1,1,0,0,1,0,0],
42     [1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,1,0,1,1,1,1,0,1,1],
43     [0,1,1,1,0,0,0,0,0,0,0,0,0,1,1,1,0,1,1,1,0,1,0,1],
44     [0,1,0,0,1,1,0,0,0,0,1,0,1,1,0,1,0,1,1,0,1,1,0,0],
45     [1,1,0,1,0,0,0,0,0,0,0,0,0,1,1,1,1,1,0,1,0,1,0,1],
46     [1,0,0,1,0,0,0,0,0,0,0,0,0,1,1,1,1,1,1,0,1,0,0,1],
47     [1,1,0,1,0,0,0,0,1,1,1,0,1,0,0,1,0,0,0,1,0,0,0,1],
48     [1,1,1,1,0,1,0,0,0,0,0,0,0,0,0,1,1,1,1,0,0,0,1,1],
49     [0,0,0,0,1,1,1,1,1,1,1,1,1,1,0,0,1,0,0,0,0,0,0,0],
50     [1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,1,1,0,1,1,1,1,0,1],
51     [1,1,1,1,0,0,1,1,0,0,0,0,0,0,0,1,0,0,0,1,1,1,0,1]
52 ]
53 adj = np.array(raw, dtype=int)
54 n = adj.shape[0]
55
56
57 mismatches = [(i,j) for i in range(n) for j in range(n) if adj[i,j] != adj[j,i]]
58 if mismatches:
59     print("Asymmetric entries found:", mismatches)
60     adj = np.maximum(adj, adj.T)
61     print("Matrix a u t o symmetrized via adj = np.maximum(adj, adj.T)")
62

```

```

63
64 diag_ones = np.where(np.diag(adj)!=0)[0]
65 if diag_ones.size > 0:
66     print(f"Self loop detected at indices {diag_ones.tolist()}, clearing them.")
67     np.fill_diagonal(adj, 0)
68
69
70 assert adj.shape == (n,n), "Wrong dimensions"
71 assert (adj == adj.T).all(), "Matrix must be symmetric"
72 assert np.all(np.diag(adj) == 0), "No self loops"
73 assert (adj.sum(axis=1) == 11).all(), "Each node must have degree 11"
74
75
76 pairs = [(i,j) for i in range(n) for j in range(i+1,n) if adj[i,j]==1]
77
78
79 occurrences = [(i,j,k) for (i,j) in pairs for k in (0,1)]
80 G = len(occurrences)
81
82
83 D = list(range(240))
84 avg = G / len(D)
85 max_games_per_day = int(np.ceil(avg))
86
87
88 m = pulp.LpProblem("11 Regular_Schedule ", pulp.LpMinimize)
89 m += 0
90
91
92 x = pulp.LpVariable.dicts("x", (occurrences, D), cat="Binary")
93
94
95 day_occ = {
96     occ: pulp.LpVariable(f"day_{occ[0]}_{occ[1]}_{occ[2]}", lowBound=0, upBound=239)
97     for occ in occurrences
98 }
99
100
101 h = {
102     occ: pulp.LpVariable(f"h_{occ[0]}_{occ[1]}_{occ[2]}", cat="Binary")
103     for occ in occurrences
104 }
105
106
107
108 for occ in occurrences:
109     m += pulp.lpSum(x[occ][d] for d in D) == 1
110
111
112 for occ in occurrences:
113     m += day_occ[occ] == pulp.lpSum(d * x[occ][d] for d in D)
114
115
116 for t in range(n):
117     occs_t = [o for o in occurrences if t in o[:2]]
118     for d in D:
119         m += pulp.lpSum(x[o][d] for o in occs_t) <= 1
120
121
122 for (i,j) in pairs:
123     m += h[(i,j,0)] + h[(i,j,1)] == 1
124
125
126 for (i,j) in pairs:
127     m += day_occ[(i,j,0)] + 1 <= day_occ[(i,j,1)]
128
129
130 WINDOW = 5
131 for t in range(n):
132     occs_t = [o for o in occurrences if t in o[:2]]
133     for start in range(len(D) - WINDOW + 1):
134         days = D[start:start+WINDOW]
135         m += pulp.lpSum(x[o][d] for o in occs_t for d in days) <= 1

```

```

136
137
138 for d in D:
139     m += pulp.lpSum(x[occ][d] for occ in occurrences) <= max_games_per_day
140
141
142 solver = pulp.PULP_CBC_CMD(msg=True, timeLimit=300)
143 m.solve(solver)
144 print("Status:", pulp.LpStatus[m.status])
145
146
147 schedule = []
148 for (i,j,k) in occurrences:
149     d = int(pulp.value(day_occ[(i,j,k)])) + 1
150     home = i if pulp.value(h[(i,j,k)]) > 0.5 else j
151     away = j if home==i else i
152     schedule.append((d, names[home], names[away], k+1))
153
154 df = pd.DataFrame(schedule, columns=["Day", "Home", "Away", "Meeting#"])
155 df.sort_values("Day").to_csv("final_schedule_11reg_24teams.csv", index=False)
156 print("Wrote final_schedule_11reg_24teams.csv")
157
158
159 country_raw = [
160     "Italy", "Poland", "Netherlands", "Serbia", "USA",
161     "Canada", "Dominican Republic", "Puerto Rico",
162     "Brazil", "Argentina", "Colombia", "Peru",
163     "Australia 1", "Australia 2", "T rkiye", "China",
164     "Japan", "Thailand", "Kenya", "Cameroon", "Germany", "Cuba", "Egypt", "Algeria"
165 ]
166 country_map = {i: country_raw[i] for i in range(n)}
167
168
169 schedule = []
170 for (i, j, k) in occurrences:
171     d = int(pulp.value(day_occ[(i, j, k)])) + 1
172     home = i if pulp.value(h[(i, j, k)]) > 0.5 else j
173     away = j if home == i else i
174     schedule.append((d, country_map[home], country_map[away], k + 1))
175
176
177 df = pd.DataFrame(schedule, columns=["Day", "HomeCountry", "AwayCountry", "Meeting#"])
178 df = df.sort_values("Day")
179 df.to_csv("final_schedule_by_country.csv", index=False)
180 print("Wrote final_schedule_by_country.csv")

```

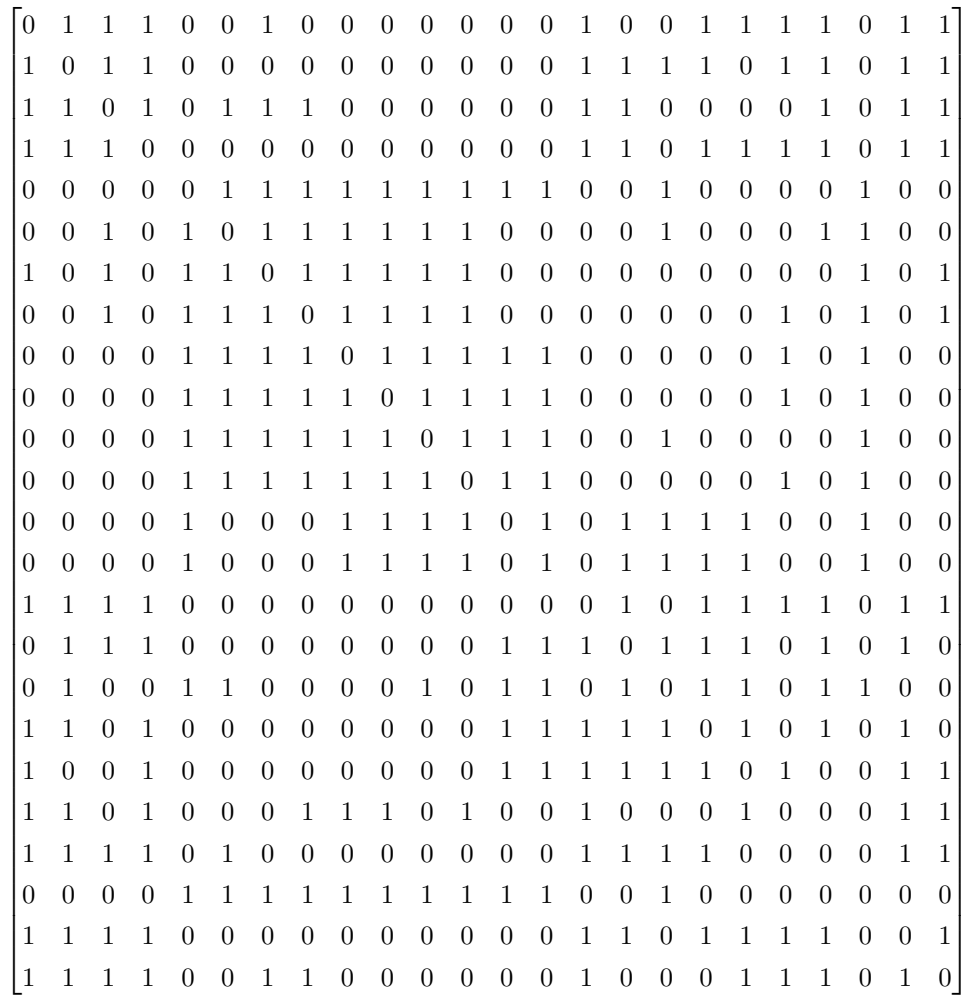


Figure 16: The transition matrix for $k=11$

Table 14: 24 Selected Teams for the Global Sports League

Day	Home Country	Away Country	Occurrence
1	Peru	USA	1
2	Egypt	Poland	1
2	Netherlands	Canada	1
3	Cameroon	TÃ¼rkiye	1
3	Australia 1	Thailand	1
4	Germany	Algeria	1
5	Dominican Republic	Colombia	1
6	Japan	USA	1
6	Brazil	Australia 2	1
7	Argentina	Puerto Rico	1
7	Canada	Netherlands	2
8	Peru	Australia 1	1
8	Poland	Egypt	2
10	TÃ¼rkiye	Germany	1
10	Kenya	China	1
11	Colombia	Japan	1
12	Italy	Thailand	1
12	Serbia	Netherlands	1
13	Argentina	Australia 2	1
13	Canada	Peru	1
15	Puerto Rico	Cameroon	1
16	Brazil	USA	1
16	Algeria	TÃ¼rkiye	1
17	Thailand	Italy	2
17	Poland	Netherlands	1
18	Cuba	Argentina	1
19	Egypt	Serbia	1
19	Dominican Republic	Peru	1
21	TÃ¼rkiye	Algeria	2
21	Puerto Rico	Canada	1
22	USA	Colombia	1
23	Germany	Netherlands	1
24	Cameroon	Argentina	1
25	Egypt	Thailand	1
26	TÃ¼rkiye	Italy	1
26	Australia 2	China	1
27	Colombia	Australia 1	1
28	Kenya	Algeria	1
29	Japan	Germany	1
29	USA	Argentina	1
30	Dominican Republic	Cuba	1

32	TÃ¼rkiye	Netherlands	1
33	Colombia	Canada	1
33	Poland	China	1
34	Egypt	Algeria	1
34	USA	Australia 1	1
36	Peru	Puerto Rico	1
37	Italy	Cameroon	1
38	Canada	Colombia	2
38	Kenya	TÃ¼rkiye	1
39	Germany	China	1
41	Cuba	Japan	1
41	Dominican Republic	USA	1
43	Cameroon	Kenya	1
43	Colombia	Australia 2	1
44	China	Australia 1	1
45	Algeria	Poland	1
46	Peru	Brazil	1
46	USA	Dominican Republic	2
47	Italy	TÃ¼rkiye	2
48	Argentina	Cuba	2
48	Thailand	Kenya	1
49	Australia 2	Japan	1
51	Puerto Rico	USA	1
52	Colombia	Dominican Republic	2
53	Germany	Poland	1
55	Peru	Cameroon	1
55	TÃ¼rkiye	Egypt	1
56	China	Netherlands	1
58	Italy	Poland	1
60	Australia 1	USA	2
61	Netherlands	Dominican Republic	1
61	Japan	Canada	1
62	Serbia	TÃ¼rkiye	1
63	Brazil	Cuba	1
63	Australia 2	Colombia	2
64	Italy	Germany	1
64	Algeria	Puerto Rico	1
66	Poland	Japan	1
67	Netherlands	China	2
68	TÃ¼rkiye	Serbia	2
69	Puerto Rico	Argentina	2
70	Colombia	Cuba	1
70	Algeria	Egypt	2
71	Australia 1	Brazil	1

71	Cameroon	Italy	2
75	Poland	Algeria	2
75	Japan	Thailand	1
76	Colombia	Argentina	1
77	Italy	Kenya	1
80	Germany	Thailand	1
80	Australia 1	Japan	1
81	Serbia	Egypt	2
82	Peru	Argentina	1
83	Canada	Cuba	1
84	Australia 2	USA	1
85	China	Türkiye	1
85	Germany	Japan	2
87	Serbia	Cameroon	1
89	Netherlands	Algeria	1
90	Argentina	Peru	2
90	China	Japan	1
92	Cuba	Brazil	2
92	Australia 1	Colombia	2
93	Cameroon	Puerto Rico	2
95	Italy	Serbia	1
95	Kenya	Australia 2	1
97	Thailand	Australia 1	2
97	Poland	Türkiye	1
98	Netherlands	Germany	2
99	Cameroon	Peru	2
99	Dominican Republic	Brazil	1
100	Kenya	Egypt	1
101	Japan	China	2
102	Italy	Algeria	1
104	Thailand	Germany	2
105	Cuba	Colombia	2
106	Türkiye	Poland	2
107	Puerto Rico	Dominican Republic	1
108	Italy	Egypt	1
110	Peru	Colombia	1
111	Serbia	Germany	1
112	Puerto Rico	Netherlands	1
113	USA	Japan	2
115	Brazil	Australia 1	2
116	Algeria	Kenya	2
116	Canada	Dominican Republic	1
117	China	Serbia	1
117	Netherlands	Poland	2

118	Cuba	Puerto Rico	1
118	Egypt	TÃ¼rkiye	2
121	Cameroon	Algeria	1
121	USA	Canada	1
123	Serbia	Poland	1
124	Germany	TÃ¼rkiye	2
124	Italy	Dominican Republic	1
128	Thailand	China	1
128	Australia 2	Peru	1
129	Puerto Rico	Brazil	1
130	Germany	Egypt	1
130	Netherlands	Serbia	2
131	Colombia	USA	2
133	Thailand	TÃ¼rkiye	1
134	Cameroon	Brazil	1
134	Japan	Kenya	1
136	Argentina	Dominican Republic	1
137	Puerto Rico	Cuba	2
137	China	Poland	2
138	Thailand	Serbia	1
139	Australia 1	Australia 2	1
140	Brazil	Canada	1
141	Argentina	Cameroon	2
142	Algeria	Italy	2
145	Australia 2	Kenya	2
146	Cuba	Australia 1	1
146	Thailand	Egypt	2
147	Colombia	Puerto Rico	1
148	Canada	Germany	1
149	Peru	Dominican Republic	2
150	TÃ¼rkiye	Cameroon	2
151	Egypt	Italy	2
152	Brazil	Puerto Rico	2
152	Japan	Australia 2	2
153	Argentina	Australia 1	1
154	Poland	Serbia	2
154	USA	Cuba	1
156	China	Kenya	2
157	Cameroon	Egypt	1
158	Italy	Netherlands	1
160	Japan	Poland	2
160	USA	Australia 2	2
161	Brazil	Peru	2
162	Kenya	Serbia	1

162	Argentina	Colombia	2
163	Egypt	China	1
163	Germany	Canada	2
165	Cameroon	Poland	1
165	Australia 2	Cuba	1
166	Australia 1	Peru	2
166	Algeria	Dominican Republic	1
167	Argentina	Brazil	1
167	Thailand	Japan	2
171	Serbia	Kenya	2
171	Puerto Rico	Peru	2
172	Australia 2	Brazil	2
172	Egypt	Netherlands	1
173	Poland	Thailand	1
175	Algeria	Germany	2
175	Australia 1	China	2
176	Peru	Cuba	1
178	Netherlands	TÃ¼rkiye	2
178	Argentina	Canada	1
179	Dominican Republic	Puerto Rico	2
179	Kenya	Cameroon	2
180	Japan	Australia 1	2
180	Thailand	Australia 2	1
181	Germany	Italy	2
182	Serbia	China	2
183	Cuba	Canada	2
184	Egypt	Cameroon	2
186	Brazil	Colombia	1
187	TÃ¼rkiye	Thailand	2
187	Puerto Rico	Algeria	2
188	Dominican Republic	Canada	2
190	USA	Peru	2
192	Puerto Rico	Colombia	2
192	Netherlands	Egypt	2
193	Poland	Italy	2
193	China	Germany	2
194	Canada	Brazil	2
196	Serbia	Thailand	2
196	Cuba	Dominican Republic	2
197	Algeria	Cameroon	2
199	China	Australia 2	2
199	Canada	Argentina	2
201	Colombia	Brazil	2
202	Kenya	Australia 1	1

204	Australia 2	Argentina	2
205	Cuba	USA	2
205	Serbia	Algeria	1
208	Kenya	Thailand	2
208	Netherlands	Italy	2
209	Peru	Canada	2
209	Poland	Cameroon	2
210	Germany	Serbia	2
210	Brazil	Argentina	2
213	Japan	Cuba	2
213	Türkiye	Kenya	2
214	Australia 2	Thailand	2
214	Dominican Republic	Algeria	2
215	Poland	Germany	2
215	Argentina	USA	2
219	Japan	Colombia	2
219	Australia 1	Kenya	2
220	Algeria	Serbia	2
221	Canada	Puerto Rico	2
221	Brazil	Dominican Republic	2
222	China	Egypt	2
223	Cuba	Peru	2
223	Thailand	Poland	2
224	Australia 2	Australia 1	2
224	Kenya	Italy	2
226	Canada	USA	2
226	Dominican Republic	Netherlands	2
228	Cameroon	Serbia	2
229	Egypt	Kenya	2
229	Australia 1	Argentina	2
231	USA	Brazil	2
231	Netherlands	Puerto Rico	2
232	China	Thailand	2
232	Colombia	Peru	2
233	Cuba	Australia 2	2
234	Dominican Republic	Argentina	2
234	Kenya	Japan	2
235	Serbia	Italy	2
236	Brazil	Cameroon	2
237	Türkiye	China	2
237	USA	Puerto Rico	2
238	Australia 1	Cuba	2
238	Peru	Australia 2	2
239	Egypt	Germany	2

239	Canada	Japan	2
240	Dominican Republic	Italy	2
240	Algeria	Netherlands	2

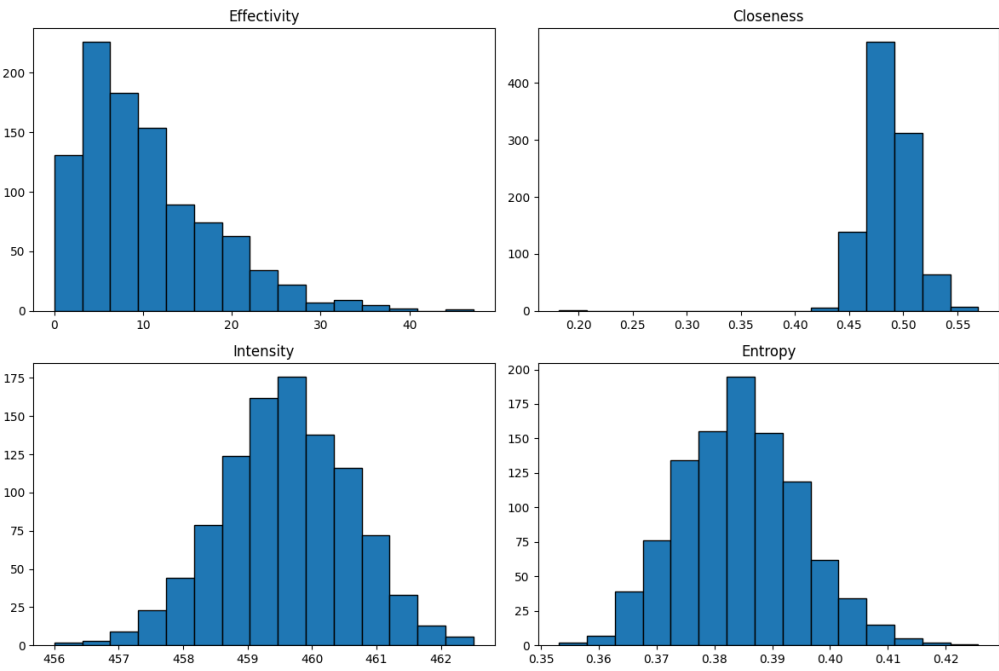


Figure 17: Swiss Results (24 teams)
Note: There is an outlier for the closeness metric, but the rest is still concentrated around 0.4-0.5.

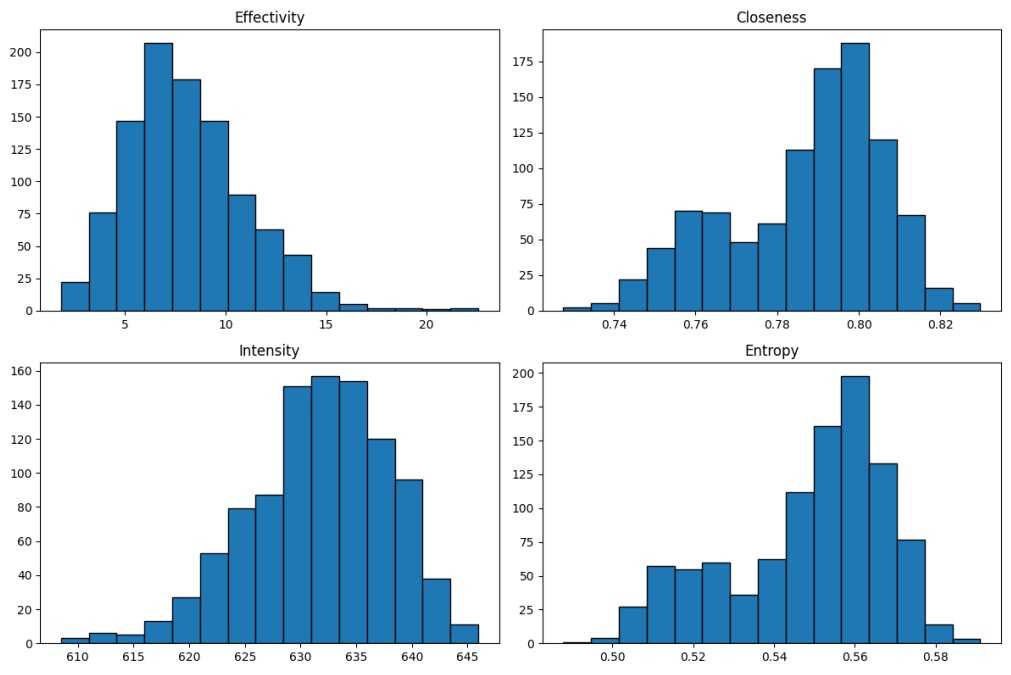


Figure 18: FIVB Schedule (Simplified)